R1.05 (partie Conception)

IUT de Bayonne et du Pays Basque – BUT Informatique

R1.05 - Introduction aux Bases de Données et SQL

Approche de la conception des bases de données

Année Universitaire 2026-2027

Agnès GASTAMBIDE et Christophe MARQUESUZAÀ

Objectifs

- Comprendre les étapes permettant de modéliser (puis manipuler) une Base de Données.
- Connaître et savoir appliquer une approche permettant de modéliser correctement une base de données au niveau conceptuel (MCD Merise ou Diagramme de Classes UML) + Commentaires + Dictionnaire des Données.
- Traduire en un modèle de niveau logique (Schéma Relationnel) + mettre à jour le Dictionnaire des Données (et les commentaires).
- Élaborer une description équivalente pour le stockage et la manipulation des données dans le système cible choisi (niveau physique en SQL).
- Savoir utiliser un Atelier de Génie Logiciel (ici *Looping*).

Plan du cours

I. Introduction
II. Le modèle E/A de Merise : concepts de base
III. Exemple
IV. Construire un Modèle Conceptuel de Données
V. Le diagramme de classes UML : concepts de base
VI. Modélisation Logique des Données
VII. Le modèle E/A de Merise et d'UML : concepts étendus

Plan du cours

I. **Introduction**
II. Le modèle E/A de Merise : concepts de base
III. Exemple
IV. Construire un Modèle Conceptuel de Données
V. Le diagramme de classes UML : concepts de base
VI. Modélisation Logique des Données
VII. Le modèle E/A de Merise et d'UML : concepts étendus

I. Introduction (1)

Qu'est-ce qu'une Base de Données

Définition 1.1 -Base de Données- ensemble organisé d'informations avec un objectif commun. Peu importe le support utilisé pour rassembler et stocker les données (papier, fichiers, etc.), dès lors que des données sont rassemblées et stockées d'une manière organisée dans un but spécifique, on parle de **base de données**.

Plus précisément, on appelle base de données un ensemble structuré et organisé permettant le stockage de grandes quantités d'informations afin d'en faciliter l'exploitation (ajout, mise à jour, recherche de données).

Définition 1.2 -Base de Données informatisée- Ensemble structuré et cohérent de données **enregistrées sur des supports accessibles par l'ordinateur**, représentant des informations du monde réel, pouvant être interrogées et mises à jour par des utilisateurs et/ou servir une ou plusieurs applications, en parallèle, de façon optimale.

Définition 1.3 -Base de Données Relationnelle- base de données structurée suivant les principes du Modèle Relationnel.

I. Introduction (2)

Qu'est-ce qu'un SGBD *(ou sigle DBMS en anglais)*

une base de données relationnelle est un ensemble de données reliées entre elles et utilisables avec un programme particulier appelé **S**ystème de **G**estion de **B**ases de **D**onnées (***SGBD***) Relationnel :

⇒ Logiciel(s) assurant *structuration*, *stockage*, *consultation*, *mise à jour*, et *maintenance* des données d'une **B**ase de **D**onnées (***BD***) relationnelle.

Exemples : **Oracle**, MySQL, PostgreSQL, MS SQL Server…

Qu'est-ce que SQL *(sigle de **S**tructured **Q**uery **L**anguage)*

un langage informatique normalisé permettant la définition, la manipulation, l'interrogation et le contrôle d'une base de données relationnelle.

I. Introduction (3)

- **Applications utilisant des bases de données**
  - Gestion du personnel, étudiants, cours, inscriptions...
  - Système de réservation de places d'avion, de train…
  - Gestion des comptes clients des banques,
  - Gestion des lignes de bus de la ville, cartographie
  - E-commerce,
  - Gestion de production, vente des entreprises
  - ...

I. Introduction (4)

- **Modélisation**

Activité qui consiste à produire un modèle.

Un **modèle** est ce qui sert ou doit servir d'objet d'imitation pour faire ou reproduire quelque chose.

On s'intéresse dans ce cours à la **modélisation des données d'un Système d'Information (SI)** qui seront stockées dans une **Base de Données**.

Objectif : Décrire les caractéristiques du Système d'Information et des données sans faire référence à une solution informatique particulière.

=> **Modélisation conceptuelle**

produire un Modèle Conceptuel de Données

I. Introduction (5)

Modèle Conceptuel de Données (MCD)

- décrit de façon formelle les données gérées dans un système d'information et les liens entre elles sans tenir compte des choix d'organisation, d'automatisation, ou techniques.
- décrit les choix de gestion en précisant la signification des invariants, leur structure et leurs liens.
- exprime le **QUOI** sur les données.

Le MCD fournit une image invariante
d'un **S**ystème d'**I**nformation *en termes de données* sans faire référence à une solution informatique particulière

I. Introduction (6)

Modèle Conceptuel de Donnée (MCD)

- Modèles les plus couramment utilisés :
  - le modèle **Entité-Association** de **Merise** (*cf.* historique de Merise *en section 6a*) et l'un de ses dérivés,
  - le **diagramme de classes d'UML** (*cf.* historique d'UML *en section 6b*).
- Ces deux modèles fournissent l'ensemble des **concepts** ainsi que les **règles d'utilisation** de ces concepts qui permettront de décrire les données d'un Système d'Information sous la forme d'un **Modèle** ou **Schéma des Données de niveau conceptuel**.

Le Modèle Conceptuel de Données (**MCD**) est une **abstraction** (un modèle **abstrait**), tandis qu'un **S**chéma **C**onceptuel des **D**onnées (SCD), sous la forme d'un *schéma Entité-Association* ou d'un *diagramme de classes UML*, est une **instance** possible (un modèle **concret**) du MCD (= résultat obtenu en utilisant les concepts du Modèle Conceptuel de Données).

I. Introduction (6a)

MERISE : méthode de conception, de développement et de réalisation de projets informatiques (**M**éthode d'**E**tude et de **R**éalisation Informatique pour les **S**ystèmes d'**E**ntreprise)

Historique Merise, Modèle Entité/Association et SQL

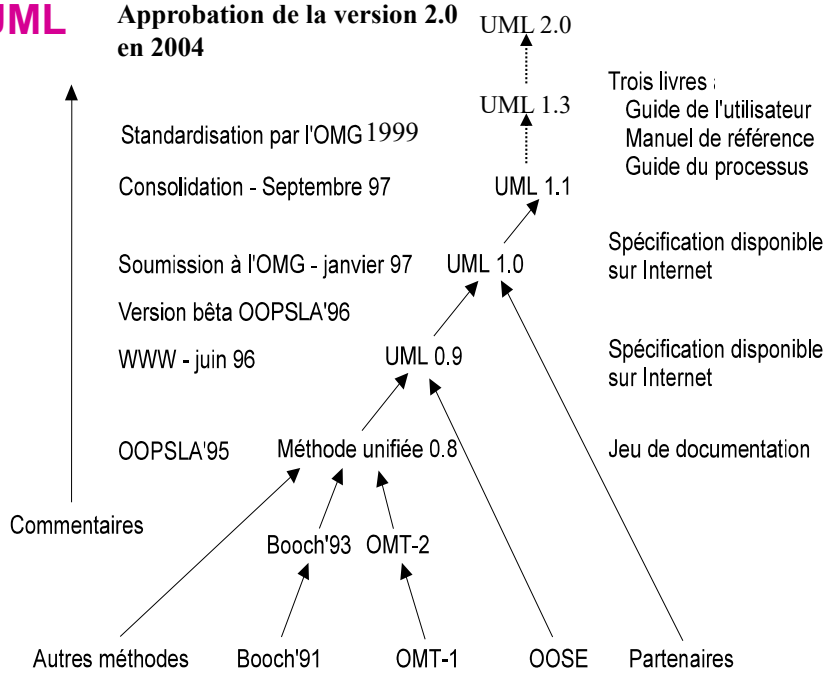
1970	**Modèle Relationnel de Codd**
Années 70	Premiers prototypes de SQL
1976	**Modèle Entité-Relation de Peter Chen (et Modèle Entité Association de P. Moulin puis Hubert Tardieu)**
1974-78	Le noyau de MERISE est établi par une équipe d'ingénieurs et de chercheurs aixois.
1978	**Développement de MERISE : méthode française de conception de systèmes d'information, sous l'égide du ministère de l'industrie.**
1979	Conception du système d'information, construction de la base de données, H. Tardieu, D. Nanci, D. Pascot (préfacé par J.-L. Le Moigne), Editions d'Organisation.
1979	Première version de **SQL**, proposé par ORACLE.
1983	La méthode MERISE - Tome 1 : principes et outils. H. Tardieu, A. Rochfeld, R. Colletti. Éditions d'Organisation.
1985	La méthode MERISE - Tome 2 : démarche et pratique. H. Tardieu, A. Rochfeld, R. Colletti, G. Panet, G. Vahée. Éditions d'Organisation.
1986	SQL ANSI (American National Standard Institute)
1989	SQL-1, ISO et ANSI (International Standard Organisation)
1989	La méthode MERISE - Tome 3 : gamme opératoire. A. Rochfeld, J. Moréjon. Édition d'Organisation.
1992	Ingéniérie des systèmes d'information : MERISE. 1ère édition. D. Nanci, B. Espinasse. Sybex.
1992	SQL-2, ISO et ANSI
fin années 90	**PHP-MySQL**
1999	SQL-3, ISO et ANSI
2001	Ingéniérie des systèmes d'information : MERISE. 4e édition. D. Nanci, B. Espinasse. Vuibert.

Le modèle **Entité/Association** a pris naissance dans des travaux réalisés en France au début des années 70 (1972-1975) par MOULIN et ALLII [Moulin et Allii, 1976] et est utilisé dans la méthode MERISE. Mais c'est à l'américain Peter CHEN qu'est attribuée la paternité du modèle (appelé modèle Entité-Relation).

En 2001, la méthode MERISE était encore la méthode de conception de systèmes d'information la plus largement pratiquée en France.

I. Introduction (6b)

Historique UML



UML (*Unified Modeling Language* ou « langage de modélisation objet unifié ») est né de la **fusion** des trois méthodes qui s'imposaient dans le domaine de la modélisation objet dans les années 1990 : OMT, Booch et OOSE.
D'importants acteurs industriels (IBM, Microsoft, Oracle, DEC, HP, Rational, Unisys etc.) s'associent alors à l'effort et proposent UML 1.0 à l'OMG (*Object Management Group*) qui l'accepte en novembre 1997 dans sa version 1.1**. La version d'UML en cours est UML 2.5.1 (Décembre 2017).**

UML s'impose plus que jamais en tant que langage de modélisation standardisé pour la modélisation des logiciels.
Le **diagramme de classes** qui modélise les classes intervenant dans le système fait partie des diagrammes structurels ou statiques d'UML. UML 2.5.1 propose 14 types de diagrammes dont 7 structurels et 7 comportementaux.

I. Introduction (7)

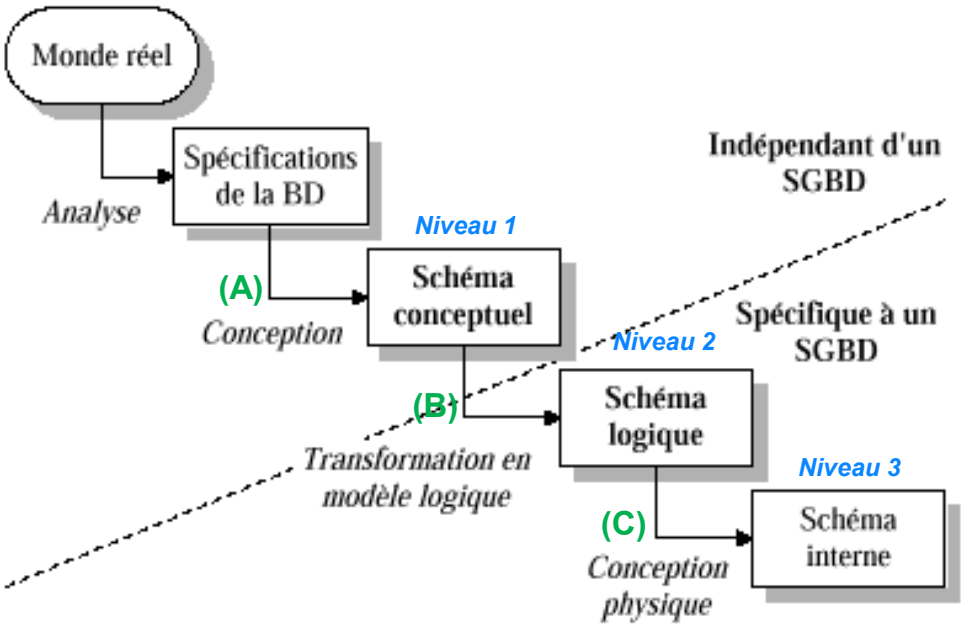
Conception d'une BD => modélisation à 3 niveaux

- Le **Modèle Conceptuel des Données** (MCD) permet de concevoir une organisation des données au sein d'une future base de données. Cette conception se veut dégagée des contraintes de mise en œuvre. Ce modèle se concrétise par un schéma basé sur un formalisme proposé par Merise (modèle E/A) ou UML (diagramme de classes).
- Le **Modèle Logique** est une transcription (également appelée dérivation) du MCD dans un formalisme adapté à une implémentation ultérieure. Le modèle logique utilisé est le **Modèle Relationnel**, modèle mathématique, qui définit des relations entre les données et des opérations sur les relations, ce modèle est à l'origine des **bases de données relationnelles**.
- Le **Modèle Physique** décrit la façon dont les données seront effectivement stockées dans la base de données qui sera gérée dans le Système de Gestion de Bases de Données choisi (ORACLE, MySQL, PostgreSQL, MS SQL-Server…).

=> Trois étapes dans la construction d'une base de données.

I. Introduction (8a)

Processus de conception d'une BD : 3 niveaux



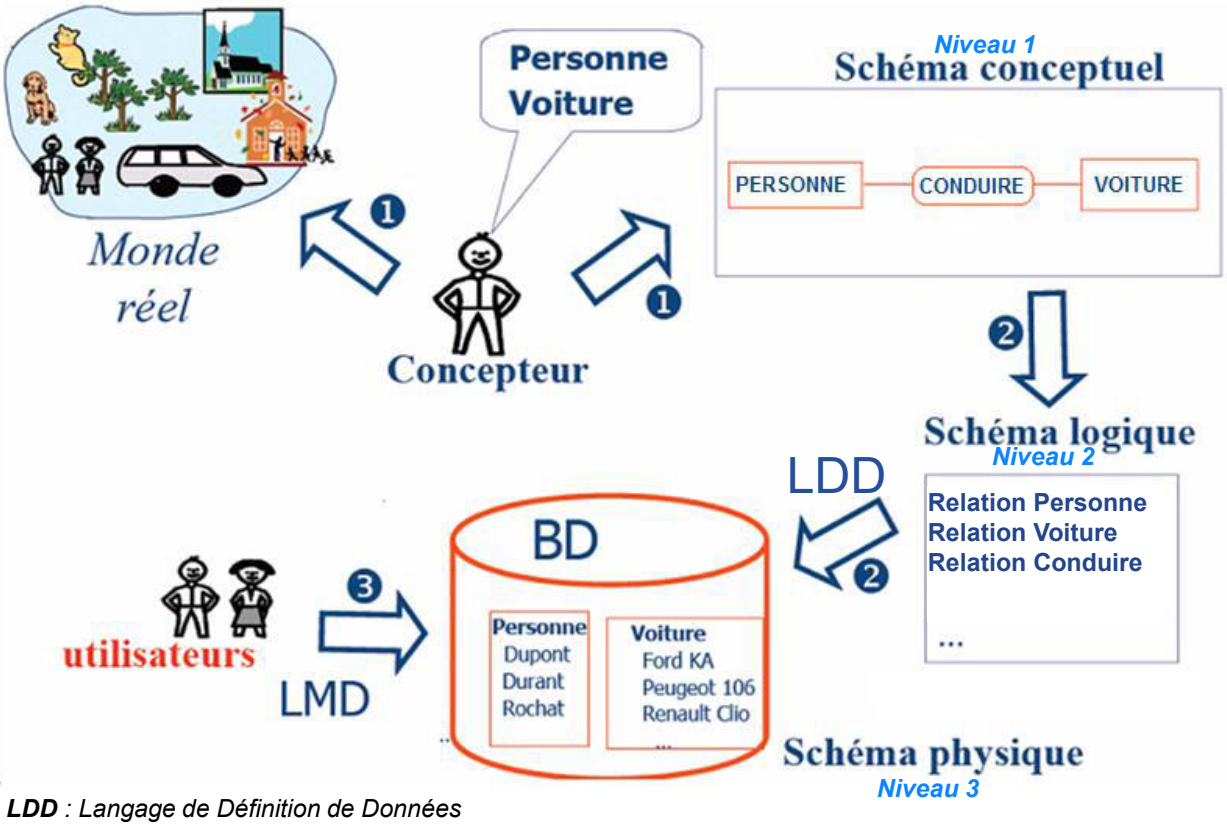
(A) R1.05 (partie Conception chapitres I à V et VII)

(B) Règles de traduction pour passer du Modèle Conceptuel au Modèle Logique c'est-à-dire du Schéma Entité/Association au Schéma Relationnel R1.05 (partie Conception chapitre VI)

(C) Implantation du Modèle Logique (Relationnel) en Modèle Physique ou Interne R1.05 (partie Manipulation)

I. Introduction (8b)

Processus de conception d'une BD : 3 niveaux



LDD *: Langage de Définition de Données*
LMD *: Langage de Manipulation de Données*

I. Introduction (9a)

Conception d'une base de données : exemple

Spécifications

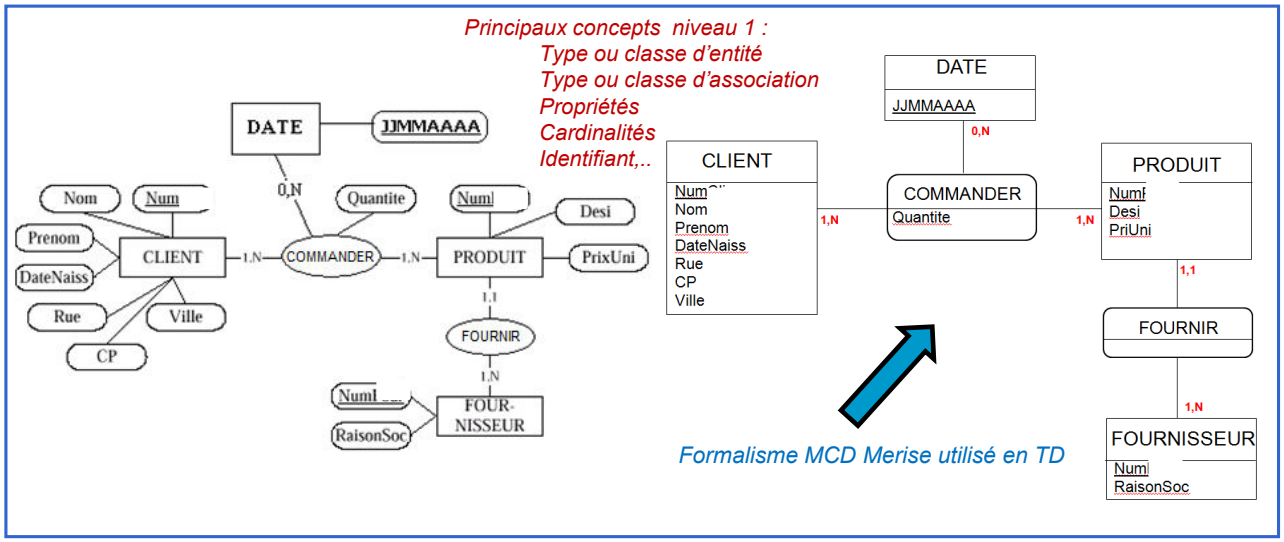
- Les **clients** sont caractérisés par un numéro de client, leur nom, prénom, date de naissance, rue, code postal et ville.
- Ils **commandent** des produits à une date donnée et dans une quantité donnée.
- Les **produits** sont caractérisés par un numéro de produit, leur désignation et leur prix unitaire.
- Chaque produit est **fourni** par un fournisseur unique (mais un fournisseur peut fournir plusieurs produits).
- Les **fournisseurs** sont caractérisés par un numéro de fournisseur et leur raison sociale.

Comment structurer ces données ? Pour pouvoir les stocker, traiter et les utiliser.

I. Introduction (9b)

Conception d'une base de données : exemple

Niveau 1 : Exemple de schéma Entité/Association => Modèle Conceptuel



Deux formalismes graphiques *pour le même schéma Entité/Association.*

Chaque schéma doit être complété par une description textuelle : dictionnaire des données, règles de gestion, ainsi que tout commentaire pertinent.

I. Introduction (9c)

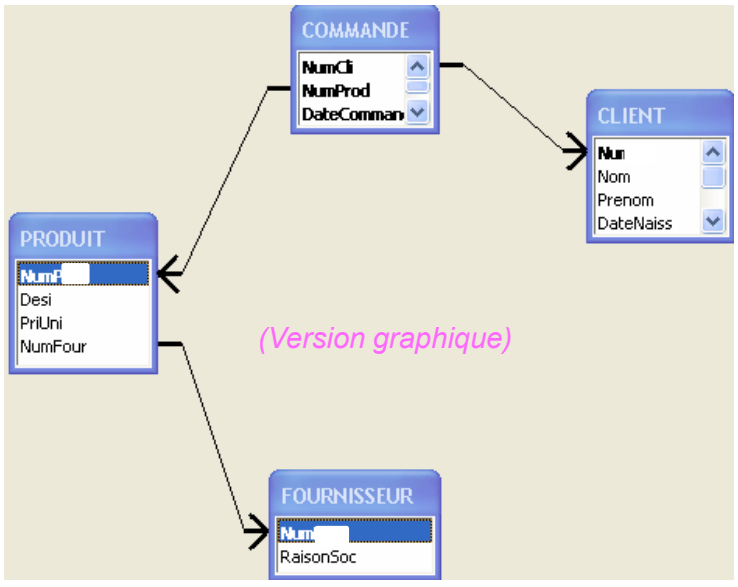
Conception d'une base de données : exemple

Niveau 2 : Exemple de Modèle logique => schéma relationnel

CLIENT (num, nom, prenom, dateNaiss, rue, CP, ville)
FOURNISSEUR (num, raisonSoc)
PRODUIT (num, desi, priUni, #numFour)
COMMANDE (#numCli, #numProd, dateCommande, quantite)

(Version textuelle)

Légende :
Clé primaire : soulignée
Clé étrangère : précédée de #



Principaux concepts niveau logique :
Relation,
Attribut,
Clé primaire,
Clé étrangère,
Domaine…

Ces deux présentations d'un schéma relationnel sont à compléter par une description textuelle comportant :
- *dictionnaire des données,*
- *règles de gestion, ainsi que*
- *tout commentaire pertinent.*

I. Introduction (9d)

Conception d'une base de données : exemple

Niveau 3 : Script SQL de création du schéma de base de données => Modèle physique

```
CREATE TABLE CLIENT (
                num INTEGER PRIMARY KEY,
                nom VARCHAR(30) NOT NULL,
                prenom VARCHAR(30),
                dateNaiss DATE,
                rue VARCHAR(50) NOT NULL,
                CP VARCHAR(5) NOT NULL,
                ville VARCHAR(20) NOT NULL,
                CONSTRAINT date_ok CHECK (dateNaiss < CURDATE()) );
CREATE TABLE FOURNISSEUR (
                num INTEGER PRIMARY KEY,
                raisonSoc VARCHAR(30) NOT NULL UNIQUE );
CREATE TABLE PRODUIT (
                num INTEGER PRIMARY KEY,
                desi VARCHAR(30) NOT NULL UNIQUE,r
                priUni FLOAT NOT NULL,
                numFour INTEGER NOT NULL,
                FOREIGN KEY (numFour) REFERENCES FOURNISSEUR(num),
                CONSTRAINT prix_ok CHECK (priUni >0 and priUni <=10000) );
CREATE TABLE COMMANDE (
                numCli  INTEGER NOT NULL,
                numProd INTEGER NOT NULL,
                dateCommande DATE NOT NULL,
                quantite INTEGER,
                CONSTRAINT quantite_ok CHECK (quantite >0),
                PRIMARY KEY (numCli,numProd,dateCommande),
                FOREIGN KEY (numCli) REFERENCES CLIENT(num),
                FOREIGN KEY (numProd) REFERENCES PRODUIT(num) );
```

Principaux concepts niveau physique :
- *Table,*
- *Attribut ou champ,*
- *Clé primaire,*
- *Clé étrangère,*
- *Contrainte,*
- *Requêtes SQL création, insertion…*
- *Requêtes SQL interrogation…*

Créer et remplir la structure de BD pour conserver les données aussi longtemps que nécessaire, pour les interroger et les mettre à jour.

I. Introduction (10)

Schéma Conceptuel des Données (niveau 1)

- Donne une **description concise** des données, des associations et des contraintes, grâce aux **concepts** fournis par le modèle de données de haut niveau (ou abstrait) ;
- Permet de se concentrer sur les **spécifications** des propriétés des données sans se soucier des détails de leur stockage ;
- Facilite la **communication** avec les utilisateurs et les acteurs du projet étant donné qu'il ne contient pas les détails de l'implémentation ;
- Doit être **validé** de façon à ce que toutes opérations (utilisateur/application) de haut niveau identifiées puissent être spécifiées ainsi que les besoins fonctionnels puissent être satisfaits.

Plan du cours

I. Introduction
II. Le modèle E/A de Merise : concepts de base
III. Exemple
IV. Construire un Modèle Conceptuel de Données
V. Le diagramme de classes UML : concepts de base
VI. Modélisation Logique des Données
VII. Le modèle E/A de Merise et d'UML : concepts étendus

Le modèle E/A de Merise : concepts de base

Sommaire

II.1 Introduction
II.2 Entités, associations et propriétés
II.3 Types d'entité, types de propriété
II.4 Types d'association
II.5 Occurrences
II.6 Identifiants
II.7 Cardinalités

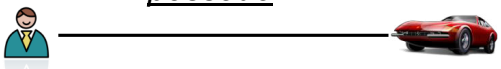
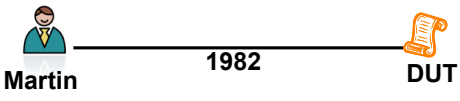
II.1. Introduction

- E/A signifie *Entité-Association*, en anglais E/R (*Entity-Relationship*).
  - *Formalisme Individu-Relation, ou*
  - *Formalisme Entity-Relationship [Chen 76]*
    ou Entité-Relation ou ***Entité-Association***
- Le modèle E/A de Merise est un *modèle conceptuel* conçu dans les années 1970.
- Il permet de construire un schéma conceptuel des données.
- Il utilise une représentation graphique.

II.2. Entités, Associations et Propriétés(1)

Concepts du modèle Entité-Association

- **Entité** : individu ou objet de l'univers du discours, concret ou abstrait, qui possède une existence intrinsèque et une certaine stabilité permettant de le repérer au cours du temps.
  Ex : l'étudiant *ETCHEVERRY*  , la voiture immatriculée *4030 NM 64*  , le diplôme *DUT*  .

- **Association** : regroupement d'entités dans lequel chaque entité joue un rôle précis.
  Ex : l'étudiant ETCHEVERRY ***possède*** la voiture immatriculée 4030 NM 64,
  
  les étudiants MARTIN ET LURO ***ont*** tous deux ***obtenu*** le diplôme DUT (respectivement en 1982 et 1985)
  

II.2. Entités, Associations et Propriétés(2)

Concepts du modèle Entité-Association

- **Propriété** : Caractéristique d'une entité ou d'une association

  Exemples :

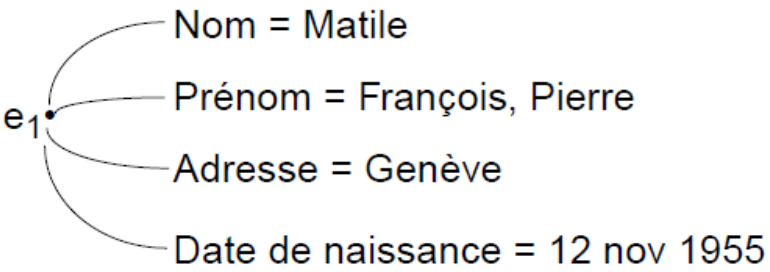
  le *département de naissance* de l'étudiant ETCHEVERRY est *64*,

  la *couleur* de la voiture immatriculée 4030 NM 64 est *rouge*,

  l'*intitulé complet* du diplôme DUT est *Diplôme Universitaire de Technologie*,

  l'année d'obtention du diplôme DUT par l'étudiant MARTIN est *1982*.

  L'entité e1 :



Une propriété est désignée par **un nom et une valeur** qui appartient à **un domaine** (défini dans le dictionnaire de données).

II.3. Types d'entité, de propriété (1)

- Type ou Classe d'entité(1) : **ensemble d'entités** définies par un même ensemble de types de propriété, jouant un rôle identique, et représentant une classe naturelle d'objets.

  ex. : les types d'entité Personne, Etudiant, Voiture, Diplome, Client,...)

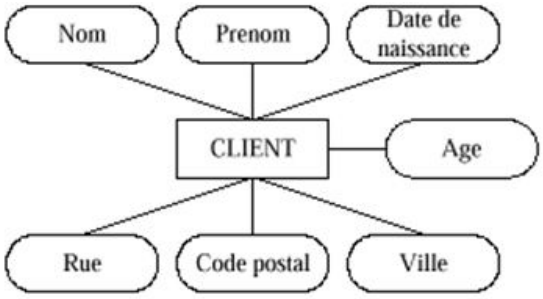
PERSONNE	ETUDIANT	DIPLOME

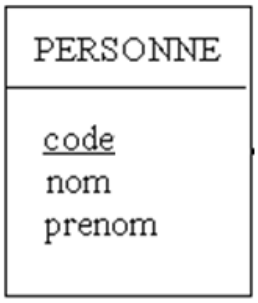
- Type de propriété(2) : Nom d'une propriété c'est-à-dire d'une **caractéristique** associée à **un type d'entité** ou à **un type d'association**

  ex. : le *département de naissance* des étudiants, la *couleur* des voitures, l'*intitulé complet* des diplômes, l'*année* d'obtention des diplômes par les étudiants, le *nom* et le *prénom* des clients, ...

(1) Type d'entité / Classe d'entité / Entité Type
(2) Type de Propriété / Type d'Attribut

II.3. Types d'entité, de propriété (2)





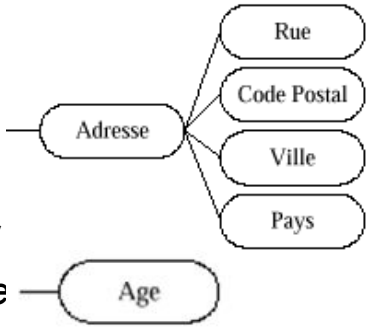
Exemples de type ou classe d'entité (Client et Personne) avec leurs types de propriété

Un type (ou une classe) d'entité est décrit par :
✓ un nom ;
✓ une liste de types de propriété ;
✓ une définition qui précise la signification que nous voulons retenir de ce type d'entité dans le cadre de la base de données.

II.3. Types d'entité, de propriété (3)

Type de propriété

- Type de propriété ***simple ou atomique*** : non divisible (ex. : Nom)
- Type de propriété ***composé/complexe*** : subdivisé en types de propriété simples (ex. : Adresse)
- *Type de propriété **dérivé** : dont la valeur est calculée (ex. : Âge calculé à partir de la date de naissance)*



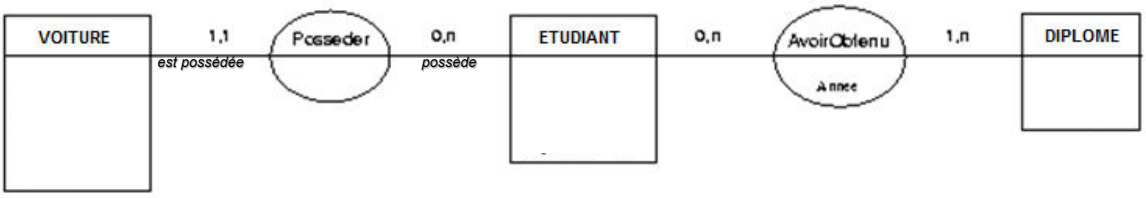
Privilégier les types de propriété simples !

II.4. Types d'association (1)

Type ou Classe d'association(1)

ensemble d'associations entre plusieurs types d'entité, ayant la même sémantique (=qui donne du sens, qui porte une signification), c'est à dire possédant les mêmes caractéristiques.

ex. : Posseder entre les types d'entité ETUDIANT et VOITURE;
AvoirObtenu entre les types d'entité ETUDIANT et DIPLOME.



Un type d'association est décrit par :
- un nom et une liste de types d'entité (non nécessairement distinctes) qui participent au type d'association avec leurs **rôles** respectifs;
Posseder(est possédée : VOITURE, possède : ETUDIANT)

- une description qui précise la signification que nous voulons retenir de ce type d'association dans le cadre de la base de données.

(1) Type d'association / Classe d'association / Association Type / Classe de relation / Type de relation

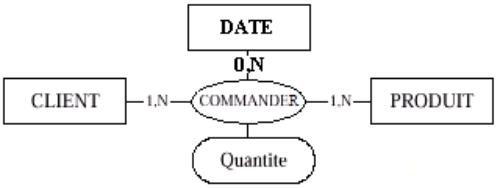
II.4. Types d'association (2)

Type de propriété pour un type (classe) d'association

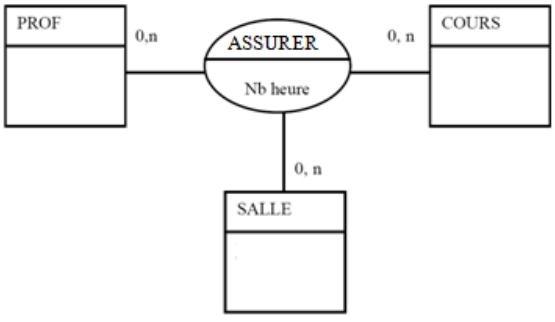
Il est possible de caractériser l'association par des types de propriété.



ex. : un étudiant a obtenu un diplôme lors d'une ***année*** donnée



ex. : une commande d'un produit effectuée par un client à une date donnée concerne une certaine ***quantité*** de produit.



ex. : un professeur donne un cours dans une salle durant un ***nombre d'heures***.

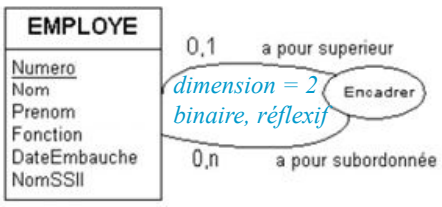
II.4. Types d'association (3)

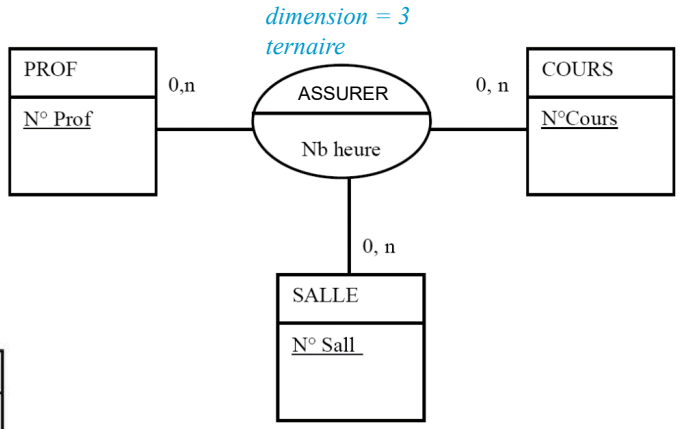
- Dimension d'un type (ou classe) d'association
  - nombre de « **pattes** » de l'association
  - type d'association ***binaire***, ***ternaire*** ou ***n-aire*** : met en jeu respectivement **deux**, **trois** ou **plusieurs** types d'entité. On dit alors qu'elle est de dimension **2**, **3** ou **n**.

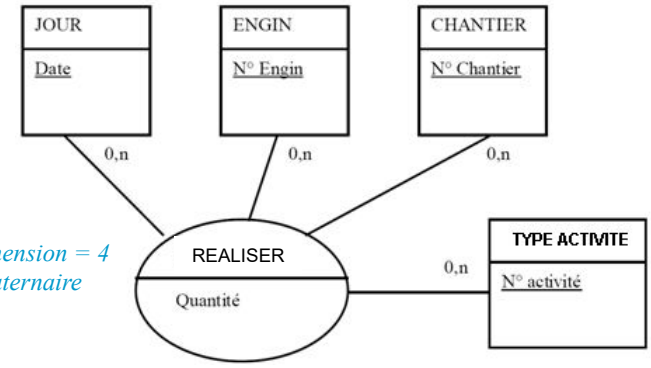
    Cas le plus fréquent : binaire
  - type d'association ***réflexif/récursif*** : met en jeu au moins deux fois le **même** type d'entité.

II.4. Types d'association (4)

- Dimension d'un type (ou classe) d'association







Exemple d'occurrence du type d'association :
Le 15/01/2009, le bulldozer F324 a réalisé sur un chantier C024 une activité de déblaiement de 300 m3



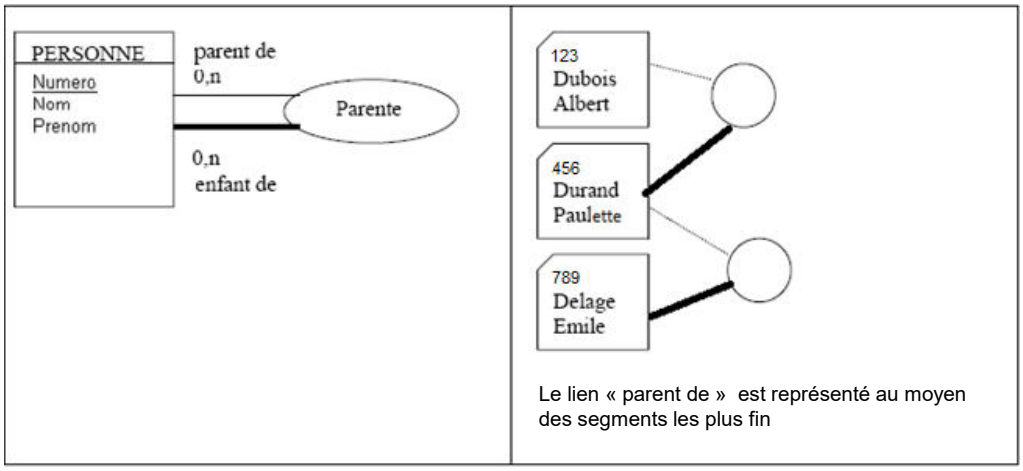
Remarque : contrairement à certains exemples de ce cours, astreignez-vous si possible à donner des noms qui soient significatifs, courts, sans espace ni caractères accentués (é, à, è, …) ou « exotiques » (ç,° …).



II.4. Types d'association (5)

- Type (ou classe) d'association réflexif/récursif

Type d'association **qui matérialise une association entre un type d'entité et lui-même**.



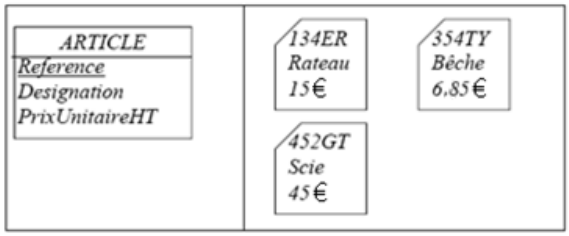
Une occurrence du type d'association établit donc un lien entre une occurrence du type d'entité et une autre occurrence de ce même type d'entité.

Dans le cas d'une association **non symétrique**, on doit faire porter le **rôle** sur chacun des segments (exemple : « *parent de* »/« *enfant de* »).

II.5. Occurrences

- Occurrence de type de propriété = valeur particulière d'un type de propriété
  ex. : Couleur. *Bleu*, *Rouge* sont des occurrences de *Couleur*.

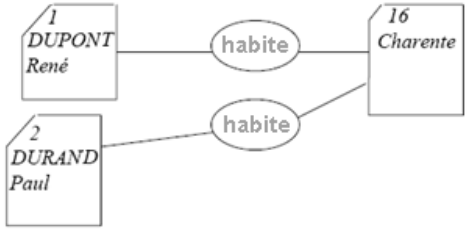
- Occurrence de type d'entité = entité ou instance
  Ex. : CLIENT a pour occurrences Albert Dupont, James West, Marie Martin, …
  Ex. : ARTICLE



- Occurrence de type d'association = lien particulier qui relie les occurrences des types d'entité qui participent à l'association.



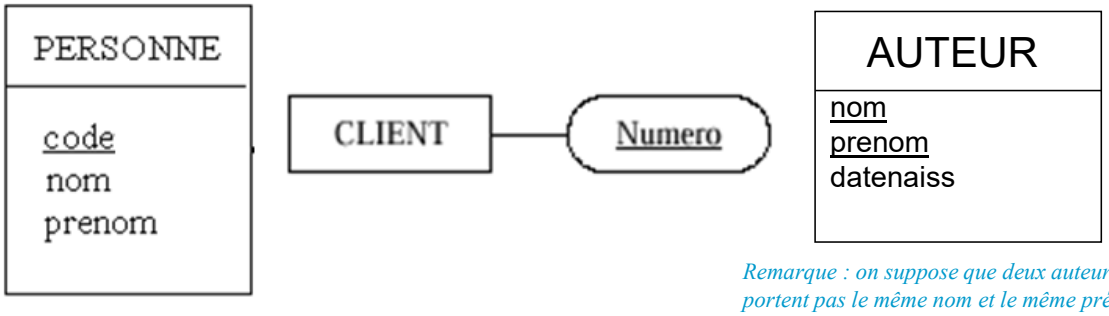
Deux exemples d'occurrence de l'association «HABITER» entre PERSONNE et DEPARTEMENT



II.6. Identifiant (1)

- **Objectif** : chaque occurrence d'un type (ou classe) d'entité doit pouvoir être repérée de manière unique et sans ambiguïté, pour être distinguée de toutes les autres.
- **Identifiant** : un **type de propriété** ou un **ensemble de types de propriété** dont la valeur permet de distinguer toutes les occurrences du type d'entité correspondant.

  Ex : numéro de client, numéro INE d'un étudiant, numéro SS d'une personne
- Caractérise **de façon *unique*** une occurrence d'un type d'entité.
- Notation graphique : les composants de l'identifiant sont **soulignés**.



Remarque : on suppose que deux auteurs ne portent pas le même nom et le même prénom

II.6. Identifiant (2)

Choix identifiant

- Si plusieurs identifiants peuvent être candidat alors choisir celui qui n'est pas composé.
- Ne pas choisir un identifiant pouvant varier au cours du temps.

  Exemple : pour un type d'entité VEHICULE, la plaque d'immatriculation.
- Si pas de candidat satisfaisant parmi les types de propriété alors en ajouter :

Exemple :

Nom	*Prenom*	*Date de Naissance*	*Etc.*
Dupont	Albert	01/06/70	...
West	James	03/09/63	...
Martin	Marie	05/06/78	...
Durand	Gaston	15/11/80	...
Titgoutte	Justine	28/02/75	...
Dupont	Noémie	18/09/57	...
Dupont	Albert	23/05/33	...

Problème : Comment distinguer les Dupont ?

II.6. Identifiant (3)

Choix identifiant

Une solution : ajouter un type de propriété *Numero*

Numero	*Nom*	*Prenom*	*Date de Naissance*
1	Dupont	Albert	01/06/70
2	West	James	03/09/63
3	Martin	Marie	05/06/78
4	Durand	Gaston	05/11/80
5	Titgoutte	Justine	28/02/75
6	Dupont	Noémie	18/09/57
7	Dupont	Albert	23/05/33

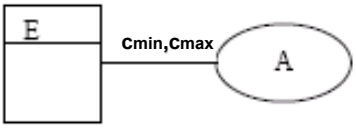
*Si parmi les propriétés du type d'entité, il n'existe pas de candidat potentiel, nous **ajouterons** un type de propriété (artificiel) dont l'unicité est garantie (code, numéro,…).*

II.7. Cardinalités (1)

- But : exprimer le nombre minimum et le nombre maximum de participations de chaque occurrence d'entité à une association.



- Cardinalités : pour un lien donné (patte), nombre **minimum** (cardinalité min.) et nombre **maximum** (cardinalité max.) d'occurrences du type d'association pouvant exister pour une seule occurrence du type d'entité.
- Lien : couple (type d'entité, type d'association)



II.7. Cardinalités (2)

La cardinalité **minimale** prend pour valeur 0 ou 1 tandis que la cardinalité **maximale** prend pour valeur 1 ou N (N pour plusieurs).

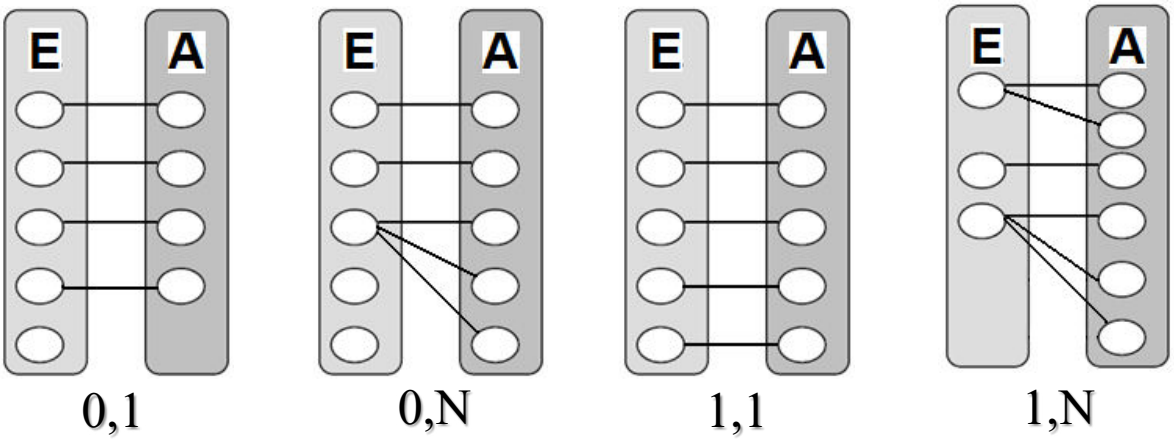
Cardinalité 0,1 : Toute occurrence du type d'entité participe **au maximum** une fois au type d'association

Cardinalité 0,N : Toute occurrence du type d'entité peut **ou non** participer au type d'association

Cardinalité 1,1 : Toute occurrence du type d'entité participe **exactement** une fois au type d'association

Cardinalité 1,N : Toute occurrence du type d'entité participe **au moins une fois** au type d'association

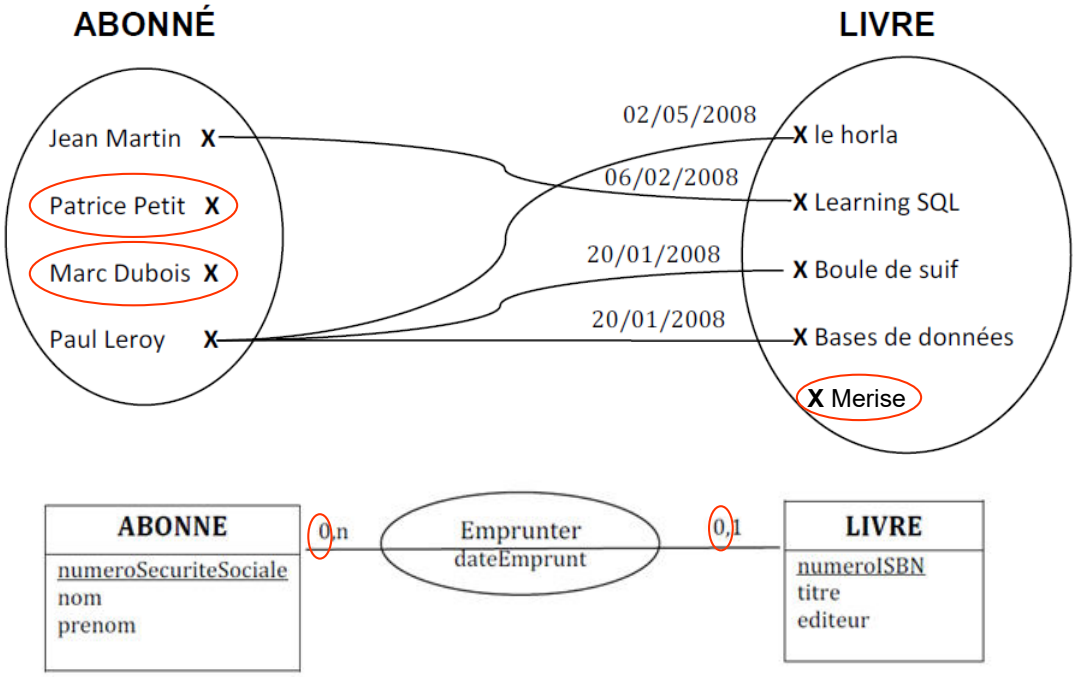
II.7. Cardinalités (3)



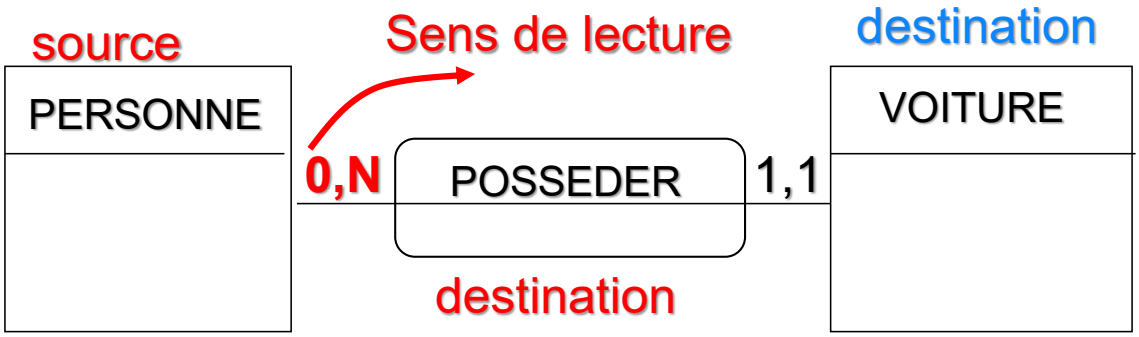
Le calcul des cardinalités se fait par rapport à la table d'occurrences du type d'association.

Remarque : dans certaines situations, la cardinalité minimale ou maximale peut être remplacée par une valeur numérique > 1 : 2, 3, …

II.7. Cardinalités (4)



II.7. Cardinalités (5)

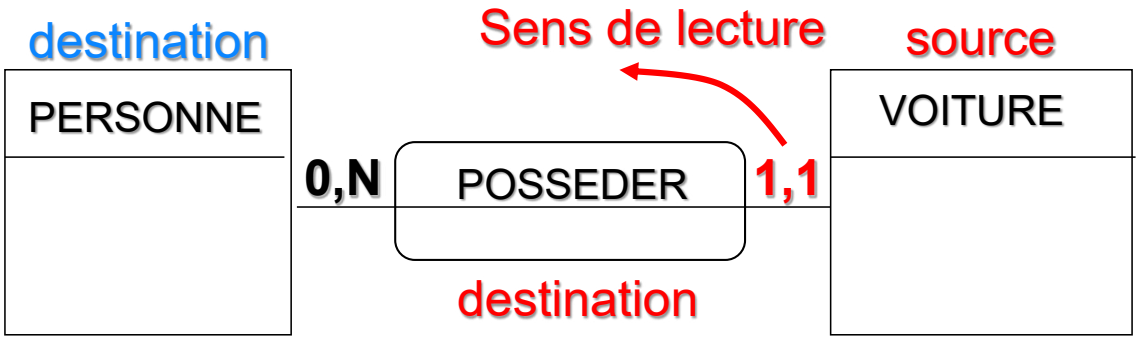


→ Première lecture : se positionner par rapport à l'association
Cardinalité 0,N : pour une occurrence de PERSONNE, combien a-t-on d'occurrences dans POSSEDER ?

- au minimum 0 occurrence (une personne peut ne pas être propriétaire de voiture);
- au maximum N occurrences (une personne peut être propriétaire plusieurs fois).

→ Autre lecture pour un type d'association **binaire**
Cardinalité 0,N : pour une occurrence de PERSONNE, combien associe-t-on d'occurrences de VOITURE ?
Une personne est propriétaire de 0 au minimum et de plusieurs voitures au maximum.

II.7. Cardinalités (6)



→ Première lecture

Cardinalité 1,1 : pour une occurrence de VOITURE, combien a-t-on d'occurrences dans POSSEDER ?

- au minimum 1
- au maximum 1

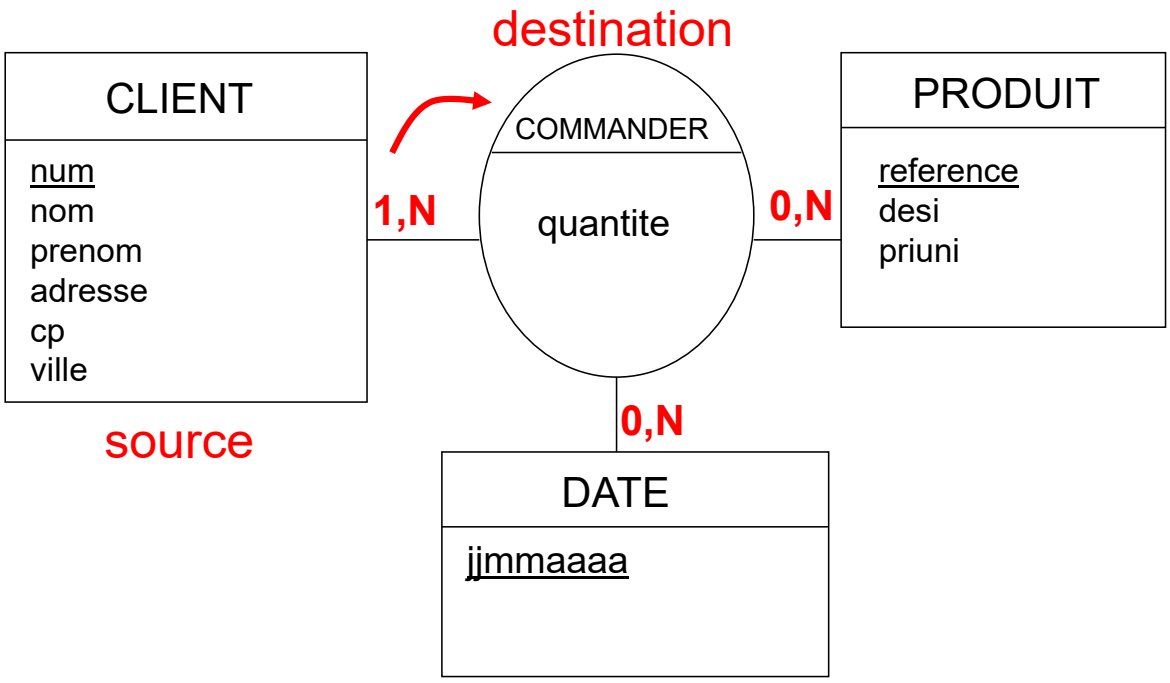
} une voiture a un et un seul propriétaire

→Deuxième lecture pour un type d'association **binaire**

Cardinalité 1,1 : pour une occurrence de VOITURE, combien associe-t-on d'occurrences de PERSONNE ?

- une et une seule

II.7. Cardinalités (7)



Cardinalité 1,N : Pour une occurrence de CLIENT, combien a-t-on d'occurrences dans COMMANDER au minimum et au maximum ? Réponse : un client commande de 1 à N fois;

Cardinalité 0,N : ……? Réponse : un produit est commandé de 0 à N fois;

Cardinalité 0,N : ……? Réponse : à une date donnée, il y a aucune ou plusieurs commandes.

II.7. Cardinalités (8)

- Un employé travaille dans un seul département



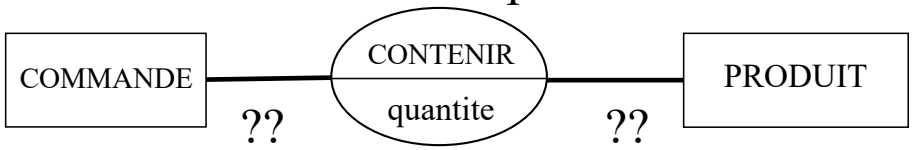
- Une commande est passée par un client



- Un employé peut diriger un département



- Une commande contient des produits



Plan du cours

I. Introduction
II. Le modèle E/A de Merise : concepts de base
III. Exemple
IV. Construire un Modèle Conceptuel de Données
V. Le diagramme de classes UML : concepts de base
VI. Modélisation Logique des Données
VII. Le modèle E/A de Merise et d'UML : concepts étendus

III. Exemple (1)

- Gestion d'une bibliothèque
  - Un client qui s'inscrit à la bibliothèque pour emprunter des livres verse une caution. Il aura le droit d'effectuer en même temps 10 emprunts au maximum.
  - Les emprunts durent au maximum 15 jours.
  - Un livre est caractérisé par son numéro dans la bibliothèque (identifiant), son titre, son éditeur et son (ses) auteurs.
  - Les auteurs sont caractérisés par leurs nom, prénom et date de naissance et chaque livre est écrit à une date donnée.
  - On veut pouvoir, pour chaque client, retrouver les emprunts effectués (nombre, numéro, titre du livre, date de l'emprunt).
  - Toutes les semaines, on édite la liste des emprunteurs en retard : nom et adresse, date de l'emprunt, numéro(s) et titre du ou des livres concernés.
  - On veut enfin pouvoir connaître pour chaque livre sa date d'achat et son état (emprunté ou non).

III. Exemple (2)

- Gestion d'une bibliothèque

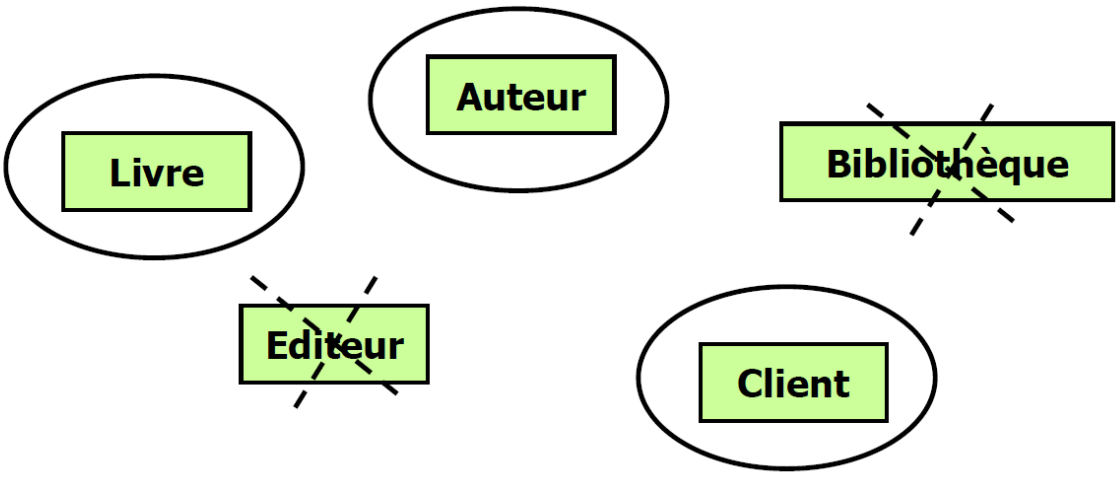
Trouver les objets ayant une existence propre

  - Un client qui s'inscrit à la bibliothèque pour emprunter des livres doit verser une caution. Il aura le droit d'effectuer en même temps 10 emprunts au maximum.
  - Les emprunts durent au maximum 15 jours.
  - Un livre est caractérisé par son numéro dans la bibliothèque (identifiant), son titre, son éditeur et son (ses) auteurs.
  - Les auteurs sont caractérisés par leurs nom, prénom et date de naissance et chaque livre est écrit à une date donnée.
  - On veut pouvoir, pour chaque client, retrouver les emprunts effectués (nombre, numéro, titre du livre, date de l'emprunt).
  - Toutes les semaines, on édite la liste des emprunteurs en retard : nom et adresse, date de l'emprunt, numéro(s) et titre du ou des livres concernés.
  - On veut enfin pouvoir connaître pour chaque livre sa date d'achat et son état (emprunté ou non).

III. Exemple (3)

- Gestion d'une bibliothèque

Trouver les objets ayant une existence propre et un intérêt pour au moins un traitement de l'application =>ce sont les types d'entité (TE)



III. Exemple (4)

- Exemple 1 : Gestion d'une bibliothèque

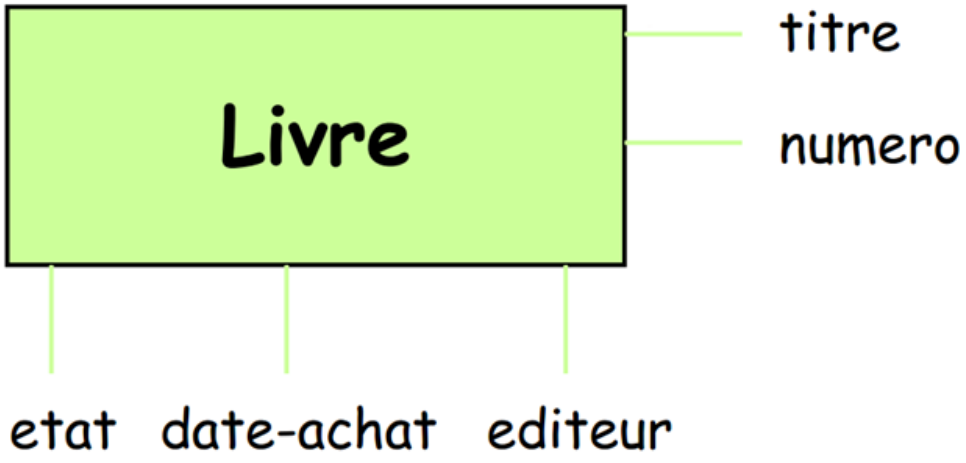
Les types de propriété du TE Livre

- Un client qui s'inscrit à la bibliothèque pour emprunter des livres doit verser une caution. Il aura le droit d'effectuer en même temps 10 emprunts au maximum.
- Les emprunts durent au maximum 15 jours.
- Un livre est caractérisé par son numéro dans la bibliothèque (identifiant), son titre, son éditeur et son (ses) auteurs.
- Les auteurs sont caractérisés par leurs nom, prénom et date de naissance et chaque livre est écrit à une date donnée.
- On veut pouvoir, pour chaque client, retrouver les emprunts effectués (nombre, numéro, titre du livre, date de l'emprunt).
- Toutes les semaines, on édite la liste des emprunteurs en retard : nom et adresse, date de l'emprunt, numéro(s) et titre du ou des livres concernés.
- On veut enfin pouvoir connaître pour chaque livre sa date d'achat et son état (emprunté ou non).

III. Exemple (5)

- Gestion d'une bibliothèque

Les types de propriété du TE Livre



III. Exemple (6)

- Gestion d'une bibliothèque

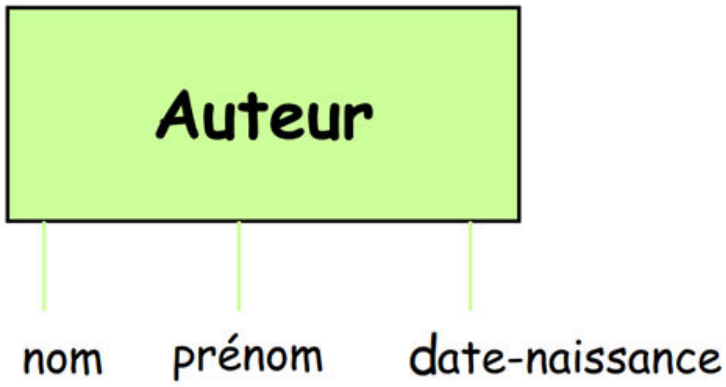
Les types de propriété du TE Auteur

- Un client qui s'inscrit à la bibliothèque pour emprunter des livres doit verser une caution. Il aura le droit d'effectuer en même temps 10 emprunts au maximum.
- Les emprunts durent au maximum 15 jours.
- Un livre est caractérisé par son numéro dans la bibliothèque (identifiant), son titre, son éditeur et son (ses) auteurs.
- Les auteurs sont caractérisés par leurs nom, prénom et date de naissance et chaque livre est écrit à une date donnée.
- On veut pouvoir, pour chaque client, retrouver les emprunts effectués (nombre, numéro, titre du livre, date de l'emprunt).
- Toutes les semaines, on édite la liste des emprunteurs en retard : nom et adresse, date de l'emprunt, numéro(s) et titre du ou des livres concernés.
- On veut enfin pouvoir connaître pour chaque livre sa date d'achat et son état (emprunté ou non).

III. Exemple (7)

- Gestion d'une bibliothèque

Les types de propriété du TE Auteur



III. Exemple (8)

- Gestion d'une bibliothèque

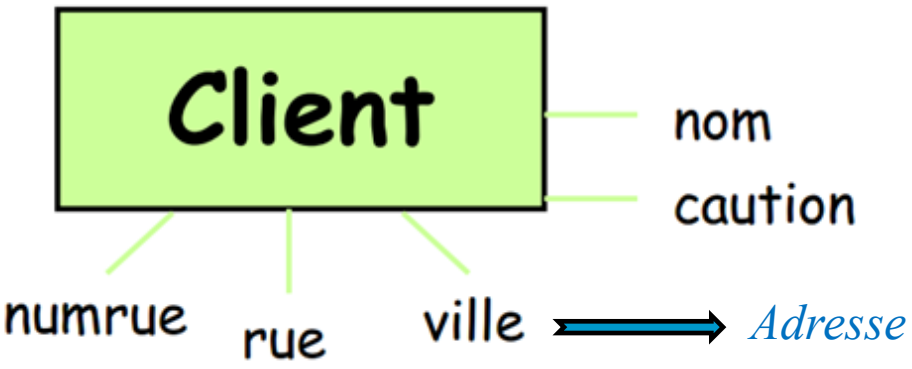
Les types de propriété du TE Client

- Un client qui s'inscrit à la bibliothèque pour emprunter des livres doit verser une caution. Il aura le droit d'effectuer en même temps 10 emprunts au maximum.
- Les emprunts durent au maximum 15 jours.
- Un livre est caractérisé par son numéro dans la bibliothèque (identifiant), son titre, son éditeur et son (ses) auteurs.
- Les auteurs sont caractérisés par leurs nom, prénom et date de naissance et chaque livre est écrit à une date donnée.
- On veut pouvoir, pour chaque client, retrouver les emprunts effectués (nombre, numéro, titre du livre, date de l'emprunt).
- Toutes les semaines, on édite la liste des emprunteurs en retard : nom et adresse, date de l'emprunt, numéro(s) et titre du ou des livres concernés.
- On veut enfin pouvoir connaître pour chaque livre sa date d'achat et son état (emprunté ou non).

III. Exemple (9)

- Gestion d'une bibliothèque

Les types de propriété du TE Client



III. Exemple (11)

- Gestion d'une bibliothèque

Les types d'entité et les types de propriété associés

LIVRE
numero titre editeur dateachat etat

CLIENT
nom numrue rue ville caution

AUTEUR
nom prenom datenaiss

III. Exemple (10)

- Gestion d'une bibliothèque

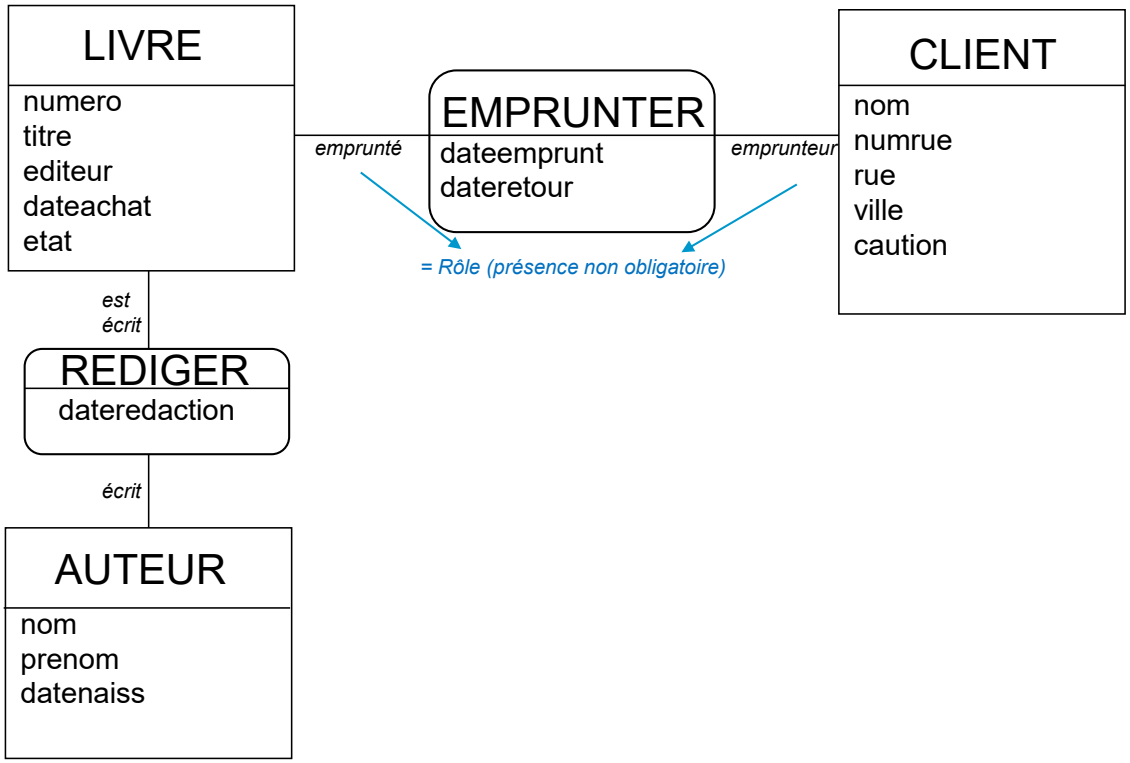
Les types d'association et les types de propriété associés

- Un client qui s'inscrit à la bibliothèque pour emprunter des livres doit verser une caution. Il aura le droit d'effectuer en même temps 10 emprunts au maximum.
- Les emprunts durent au maximum 15 jours.
- Un livre est caractérisé par son numéro dans la bibliothèque (identifiant), son titre, son éditeur et son (ses) auteurs.
- Les auteurs sont caractérisés par leurs nom, prénom et date de naissance et chaque livre est écrit à une date donnée.
- On veut pouvoir, pour chaque client, retrouver les emprunts effectués (nombre, numéro, titre du livre, date de l'emprunt).
- Toutes les semaines, on édite la liste des emprunteurs en retard : nom et adresse, date de l'emprunt, numéro(s) et titre du ou des livres concernés.
- On veut enfin pouvoir connaître pour chaque livre sa date d'achat et son état (emprunté ou non).

III. Exemple (11)

- Gestion d'une bibliothèque

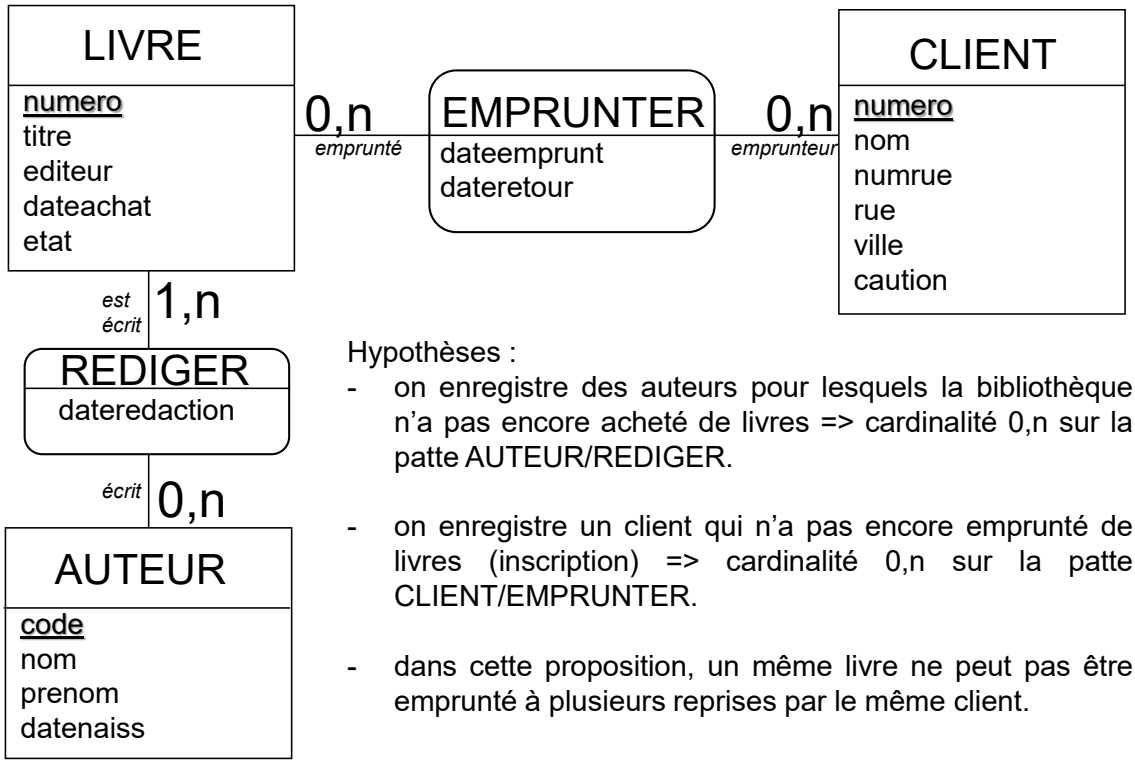
Les types d'association et les types de propriété associés



III. Exemple (12)

- Gestion d'une bibliothèque

Les identifiants et les cardinalités



Hypothèses :

- on enregistre des auteurs pour lesquels la bibliothèque n'a pas encore acheté de livres => cardinalité 0,n sur la patte AUTEUR/REDIGER.
- on enregistre un client qui n'a pas encore emprunté de livres (inscription) => cardinalité 0,n sur la patte CLIENT/EMPRUNTER.
- dans cette proposition, un même livre ne peut pas être emprunté à plusieurs reprises par le même client.
- …..

III. Exemple (13)

- Gestion d'une bibliothèque

Dictionnaire de données + commentaires

Libellé	Signification	Domaine conceptuel	logique	physique	Taille	Contraintes	Exemple
numero	Numéro de livre	Numérique			4	Identifiant unique	1234
titre	Titre du livre	Alphanumérique			100	Obligatoirement renseigné	Frappe-toi le cœur
editeur	Editeur	Alphanumérique			50	Obligatoirement renseigné	Albin Michel
….							

- Justifier l'ajout de propriétés pour définir les identifiants.
- Justifier les cardinalités.
- Autres contraintes (non représentées graphiquement)
  - ➢ Pour chaque occurrence d'Emprunter si la date **dateretour** existe, alors elle doit être supérieure à la date **dateemprunt**.
  - ➢ Pour chaque occurrence de **Livre**, la date dateachat doit être inférieure à la date **dateemprunt** de toutes les occurrences de Emprunter qui lui sont liées.
  - ➢ Un client a le droit d'effectuer en même temps 10 emprunts au maximum => Pour une occurrence de **Client**, il ne peut y avoir au maximum que 10 occurrences d'**Emprunter** pour laquelle la date **dateretour** n'est pas renseignée.
  - ➢ Les emprunts durent au maximum 15 jours.

Tables d'occurrences : tableaux d'occurrences pour chaque classe d'entité et chaque classe d'association (LIVRE, AUTEUR, CLIENT, REDIGER, EMPRUNTER)

Plan du cours

I. Introduction
II. Le modèle E/A de Merise : concepts de base
III. Exemple
IV. Construire un Modèle Conceptuel de Données
V. Le diagramme de classes UML : concepts de base
VI. Modélisation Logique des Données
VII. Le modèle E/A de Merise et d'UML : concepts étendus

IV. Construire un Modèle Conceptuel de Données (1)

La méthode présentée ici dite **descendante** consiste à partir de l'univers de discours pour réaliser le modèle (schéma).

Attention, il faut traiter le problème selon l'énoncé et non selon ses convictions personnelles.

Les étapes sont :

1. Souligner les mots clés dans le texte (l'univers de discours),
2. Identifier les types (ou classes) d'entité,
3. Identifier les types (ou classes) d'association,
4. Identifier les types de propriétés ou propriétés pertinent(e)s,
5. Placer les propriétés correspondant aux types (ou classes) d'entité,
6. Placer les propriétés correspondant aux types (ou classes) d'association,
7. Déterminer l'identifiant pour chaque type (ou classe) d'entité,
8. Établir les cardinalités sur chaque lien des types (ou classes) d'association,
9. Identifier les types (ou classes) d'association(réflexive, binaire ou n-aire),
10. Remettre en cause la modélisation obtenue (vérification des contraintes, etc…) et recommencer si besoin.

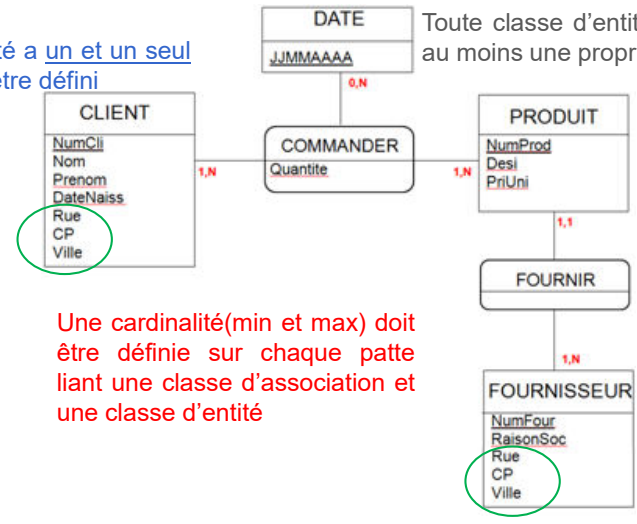
IV. Construire un Modèle Conceptuel de Données (2)

Quelques compléments

- Toute propriété doit être **élémentaire**;
- Un type (ou une classe) d'entité possède **au moins** une propriété;
- A chaque occurrence d'un type (ou classe) d'entité, il ne peut y avoir **au plus** qu'une valeur pour chaque propriété;
- Un type (ou une classe) d'entité a **un et un seul identifiant défini**;
- Un type (ou une classe) d'entité participe à **au moins** une association;
- Un type (ou une classe) d'association n'a pas d'identifiant propre mais ses occurrences seront identifiées par la **concaténation des identifiants** des entités qu'elle relie;
- Un type (ou une classe) d'association peut ne posséder **aucune propriété**;
- On ne peut pas avoir 2 fois la même propriété pour un même type (ou classe) d'entité ou pour un même type (ou classe) d'association;
- Le nom d'un type (ou classe) d'entité doit être **unique**;
- Le nom d'un type (ou classe) d'association doit être **unique**;
- …..

IV. Construire un Modèle Conceptuel de Données (3)

Toute classe d'entité a un et un seul identifiant qui doit être défini

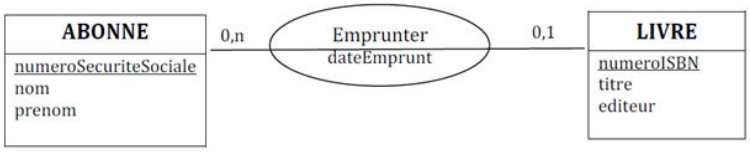


Toute classe d'entité possède au moins une propriété

Une classe d'association peut ne posséder aucune propriété

Une cardinalité(min et max) doit être définie sur chaque patte liant une classe d'association et une classe d'entité

On peut avoir 2 fois la même propriété sur des classes d'entité ou d'association distinctes.



Pas d'identifiant à définir pour une classe d'association mais un identifiant implicite(concaténation des identifiants des classes d'entité participantes)
Exemple : pour Emprunter => numeroSecuriteSociale,numeroISBN

Par commodité, les termes ***entité***, ***association***, ***propriété*** *peuvent être utilisés respectivement en lieu et place des termes* ***type (ou classe) d'entité***, ***type (ou classe) d'association*** *et* ***type de propriété****.*

IV. Construire un Modèle Conceptuel de Données (4)

- Schéma Conceptuel de Données : représentation graphique du résultat de la modélisation du Système d'Information.
- Dictionnaire des données : représentation de toutes les informations contenues dans le SCD du Système d'Information; il décrit :
  - les types (ou classes) d'entité,
  - les types (ou classes) d'association,
  - les types de propriété.

  + Règles de gestion + tout commentaire pertinent
  - les cardinalités (explications, hypothèses retenues),
  - la dimension des types (ou classes) d'association (binaire, …),
  - les dépendances entre les propriétés,
  - toute contrainte (non représentable graphiquement)
- Tables des occurrences : permettre de confirmer la définition des propriétés ainsi que le choix des cardinalités.

IV. Construire un Modèle Conceptuel de Données (5)

- Dans le dictionnaire des données du MCD, chaque type (ou classe) d'entité et chaque type (ou classe) d'association possédant des propriétés est détaillé sous la forme d'un tableau. Ce tableau comporte à minima 6 colonnes (dont l'une divisée en 3, une pour chaque niveau *conceptuel/logique/physique*) :
- Exemple pour un type d'entité Client

Libellé	Signification	Domaine			Taille	Contraintes	Exemple
		Conceptuel	Logique	Physique			
numcli	Numéro de client	numérique			4	Identifiant unique	1234
nom	Nom du client	alphanumérique			30	Obligatoirement renseigné	Dupond
datenaiss	Date de naissance du client	date				Obligatoirement renseigné < Date du jour	01/06/1970
….							

- But :
  - Préciser chaque propriété et vérifier la validité des données
  - Ultérieurement, estimer la place de stockage des données
  - Très important, si plusieurs personnes travaillent sur une même base de données, d'un volume conséquent.

IV. Construire un Modèle Conceptuel de Données (6)

Domaine conceptuel = ensemble de définition des valeurs possibles d'une propriété. On appelle cela également «Nature ou Type».

Propriété	Domaine/Type conceptuel
Nom d'une personne	Alphanumérique
Quantité en stock	Numérique (=>nombre entier positif)
Prix d'un article	Numérique (=>nombre avec 2 décimales)
Sexe	Alphabétique (=> Lettre "M" ou "F")
Date de commande	Date (format Jour/mois/année)
Jour	Alphabétique (=> Liste "Lundi", "Mardi", ...)
Age	Numérique (=> compris entre 0 et 125)

Pour les niveaux **logique** et **physique** du processus de conception, prévoir deux colonnes supplémentaires : « domaine logique » et « domaine physique ».

IV. Construire un Modèle Conceptuel de Données (7)

Exemple

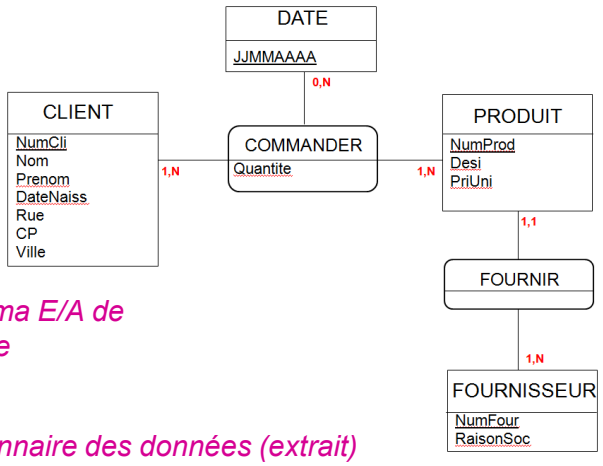


Schéma E/A de Merise

Spécifications

- Les **clients** sont caractérisés par un **numéro** de client, leur **nom**, **prénom**, **date de naissance**, **rue**, **code postal** et **ville**.
- Ils **commandent** des produits à une **date** donnée et dans une **quantité** donnée.
- Les **produits** sont caractérisés par un **numéro** de produit, leur **désignation** et leur **prix unitaire**.
- Chaque produit **est fourni** par un fournisseur unique (mais un fournisseur peut fournir plusieurs produits).
- Les **fournisseurs** sont caractérisés par un **numéro** de fournisseur et leur **raison sociale**.

Dictionnaire des données (extrait)

Libellé	Signification	Domaine Conceptuel	Logique	Physique	Taille	Contraintes	Exemple
numcli	Numéro de client	numérique			4	Identifiant unique	1234
nom	Nom du client	alphanumérique			30	Obligatoirement renseigné	Dupond
datenaiss	Date de naissance du client	date				Obligatoirement renseigné < Date du jour	01/06/1970
….							

IV. Construire un Modèle Conceptuel de Données (8)

Exemple : tables d'occurrences

But : permettre de confirmer la définition des propriétés ainsi que le choix des cardinalités

CLIENT

numcli	nom	prenom	datenaiss	rue	cp	ville
1110	Dupont	Albert	01/06/1970	3 rue des Lilas	64220	Saint Jean Pied de Port
2002	West	James	03/09/1963	33 rue de l'Ouest	64100	Bayonne
3333	Martin	Marie	05/06/1978	68 rue André Malraux	64200	Biarritz
5552	Titgoutte		28/02/1975	3 rue Pannecau	64100	Bayonne

FOURNISSEUR

numfour	raisonsoc
1	Fournisseur n°1
2	Fournisseur n°2
3	Fournisseur n°3

PRODUIT

numprod	desi	priuni
101	Raquette de tennis	900
103	Ti-shirt	200
104	Short	300
112	Chaussettes	100

DATE

jjmmaaaa
22/09/1999
02/02/2000
10/02/2000
12/03/2001
15/10/2001

Une table d'occurrences pour chaque type/classe d'entité

COMMANDER

numcli	numprod	jjmmaaaa	quantite
1110	101	22/09/1999	1
1110	101	02/02/2000	1
3333	103	22/09/1999	2
3333	112	22/09/1999	5
5552	104	10/02/2000	2
5552	112	10/02/2000	1

Une table d'occurrences pour chaque type/classe d'association

FOURNIR

numprod	numfour
101	1
103	3
104	1
112	2

Dans les tables d'occurrences des classes d'association, on retrouve en colonnes l'identifiant de chaque classe d'entité participante + les propriétés propres à l'association s'il y en a.

IV. Construire un Modèle Conceptuel de Données (9)

Dépendances fonctionnelles

- Une propriété (ou un groupe de propriétés) X **dépend fonctionnellement** d'une autre propriété (ou groupe de propriétés) Y si étant donné une **valeur de X**, il lui correspond une **valeur unique de Y**

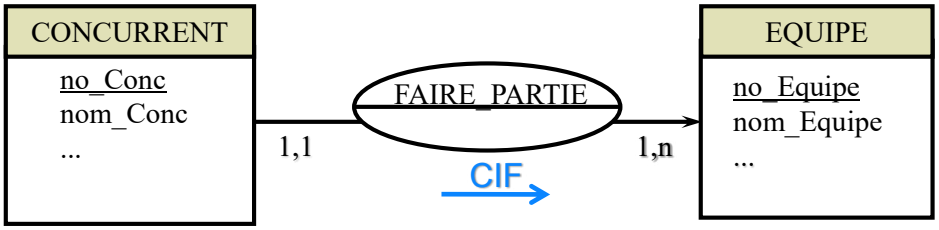
  On note **X -> Y** (X **détermine** Y)

  Cette relation est **transitive** : si X -> Y et Y -> Z alors X -> Z.

- Les propriétés non identifiantes d'une entité dépendent fonctionnellement de l'identifiant.

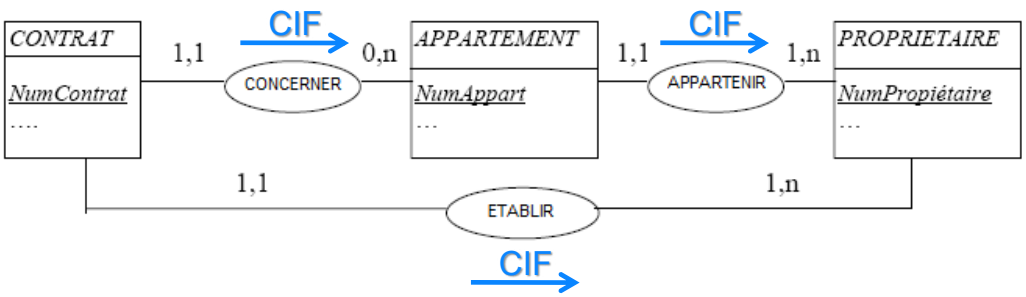
Numero	*Nom*	*Prenom*	*Date de Naissance*
1	Dupont	Albert	01/06/70
2	West	James	03/09/63

- Une cardinalité 1,1 ou 0,1 est la source d'une **dépendance fonctionnelle** : notion de CIF (**C**ontrainte d'**I**ntégrité **F**onctionnelle).



IV. Construire un Modèle Conceptuel de Données (10)

- Association transitive : toute association pouvant être obtenue par transitivité à partir de n autres associations



L'association binaire « ETABLIR » qui relie «CONTRAT» et «PROPRIETAIRE» doit être ôtée du schéma car on peut retrouver le propriétaire qui a établi le contrat à partir des associations «CONCERNER» et «APPARTENIR». Il s'agit d'une association transitive.

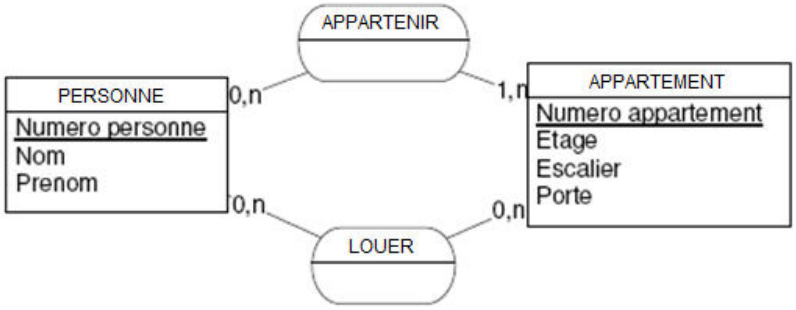
On veillera à supprimer les dépendances transitives pour ne pas surcharger le Modèle Conceptuel des Données d'une part, et d'autre part, on risque d'aboutir à des incohérences.

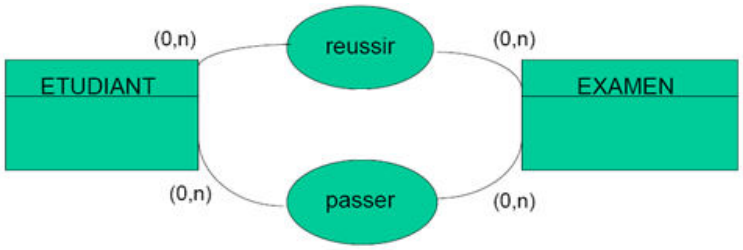
IV. Construire un Modèle Conceptuel de Données (11)

- Associations multiples

Plusieurs associations peuvent lier deux types (ou classes) d'entité à condition que la sémantique soit différente.

Exemples :





IV. Construire un Modèle Conceptuel de Données (12)

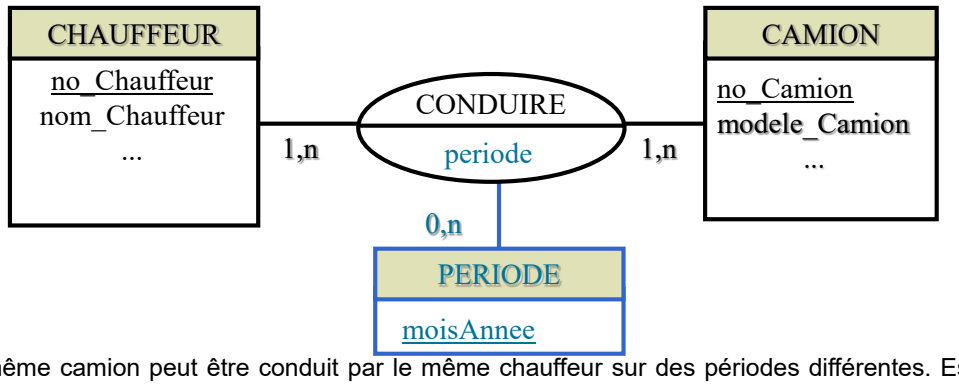
Historisation et suivi chronologique

Modification de l'information au cours de la vie du système

Historisation des associations

Ex. CHAUFFEUR-CONDUIRE-CAMION

Hypothèse : Un chauffeur se voit attribuer un camion pour une période donnée. Nous admettons que la période représente un mois calendaire (par exemple, janvier 2012).



Si le même camion peut être conduit par le même chauffeur sur des périodes différentes. Est-ce que ce schéma est satisfaisant ?

Non car,

- L'association « CONDUIRE » est une association binaire. On ne peut enregistrer qu'une seule fois une association entre un chauffeur_i avec un camion_j.
- La propriété « periode » de l'association est une propriété élémentaire donc il n'est pas possible d'enregistrer plusieurs périodes pour le même couple (chauffeur_i camion_j).

Quelle solution proposez-vous ?

Si on souhaite historiser la conduite des camions par les chauffeurs, il faut rajouter une dimension à l'association (issue de la transformation de la propriété temporelle « période » en un axe dimensionnel).

IV. Construire un Modèle Conceptuel de Données (13)

Historisation et suivi chronologique

Modification de l'information au cours de la vie du système

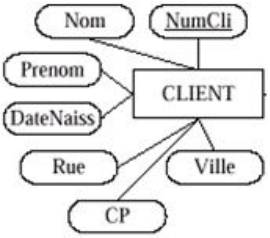
Datation

Date => type de propriété

Elle doit être considérée comme propriété d'un type (ou classe) d'entité ou d'une association si elle est unique et invariante dans l'univers du discours.

ex. date de naissance d'un client

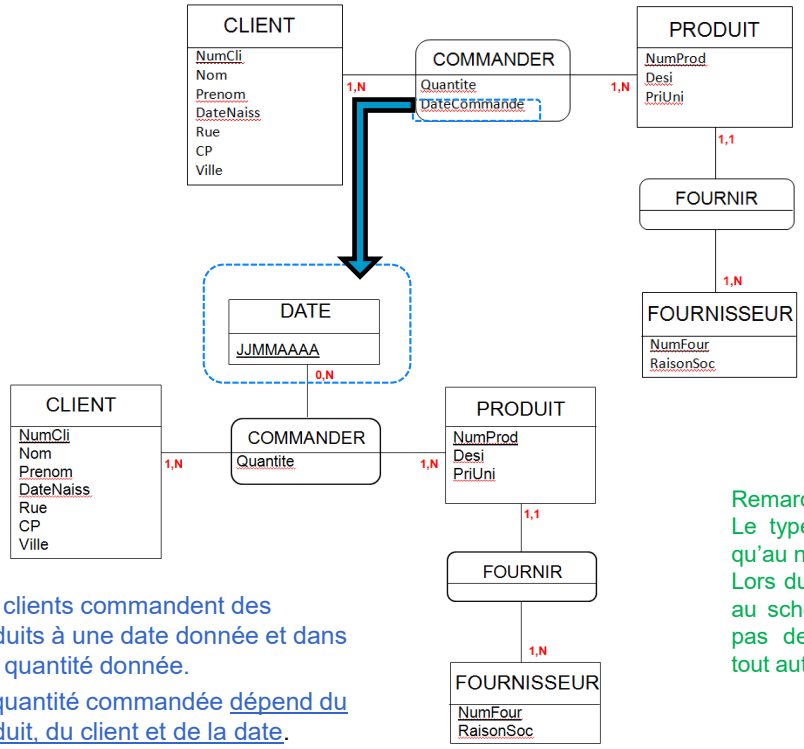




IV. Construire un Modèle Conceptuel de Données (14)

Historisation et suivi chronologique

Datation : **Date => type d'entité**



Si un même client peut commander le même produit à des dates différentes, est-ce que ce schéma est satisfaisant ? Non

Chaque occurrence de l'association binaire COMMANDER correspond à un lien unique entre une occurrence de l'entité participante CLIENT et une occurrence de l'autre entité participante PRODUIT.

Les clients commandent des produits à une date donnée et dans une quantité donnée.

La quantité commandée dépend du produit, du client et de la date.

Remarque :
Le type d'entité DATE n'a d'existence qu'au niveau conceptuel.
Lors du passage du schéma conceptuel au schéma relationnel, cet élément n'a pas de traduction en relation comme tout autre type d'entité.

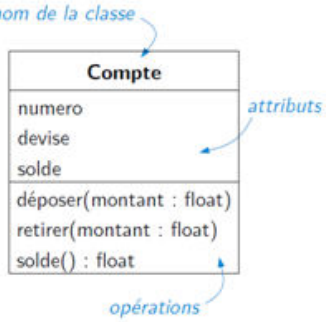
Plan du cours

I. Introduction
II. Le modèle E/A de Merise : concepts de base
III. Exemple
IV. Construire un Modèle Conceptuel de Données
V. Le diagramme de classes UML : concepts de base
VI. Modélisation Logique des Données
VII. Le modèle E/A de Merise et d'UML : concepts étendus

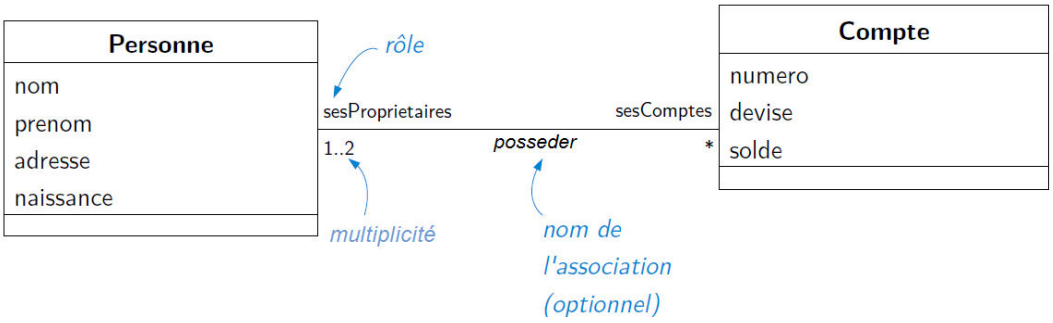
V. Le diagramme de classes UML (1)

Un diagramme de classes permet de décrire la structure **statique** du système à l'aide des concepts :

– classes (attributs, *opérations*, …)



– relations : association, classe-association, *agrégation, composition, héritage, dépendance*, etc.

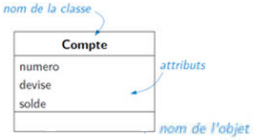


V. Le diagramme de classes UML (2)

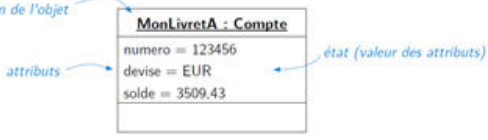
Comparaison avec le modèle Entité/Association de Merise

Équivalences des concepts de base (E/A de Merise ⇔ Diag. de classes UML) :

– Type (classe) d'entité ⇔ **Classe**

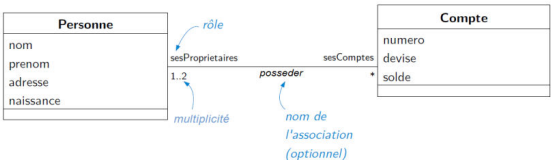


– Occurrence, entité ⇔ **Objet**, **instance**

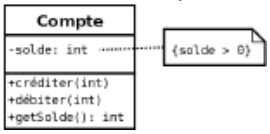


– Type (classe) d'association ⇔ **Association** / **Classe-association**

– Cardinalités ⇔ **Multiplicités**



– Contraintes ⇔ **Contraintes** (sous forme textuelle, entre accolades / Notes / Langage OCL*)



– Modèle E/A de Merise ⇔ **Diagramme de classes UML**

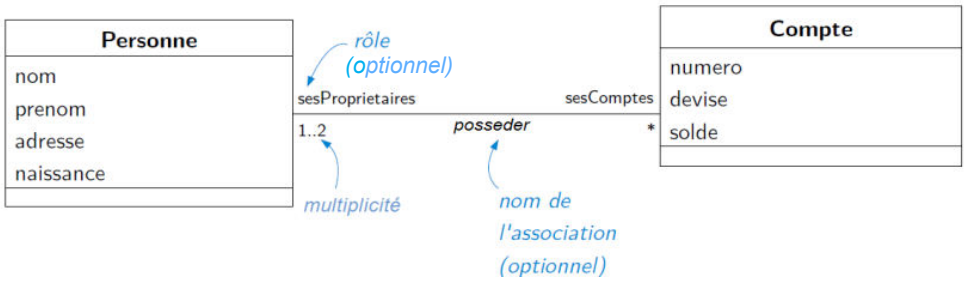
***OCL : Object Constraint Language** (langage mathématique formel)

V. Le diagramme de classes UML (3)

Comparaison avec le modèle Entité/Association de Merise

- cardinalités ⇔ multiplicités

  – 1,1 ⇔ 1 (CIF)
  – 1,N ⇔ 1..*
  – 0,1 ⇔ 0..1 (CIF)
  – 0,N ⇔ 0..* ou *



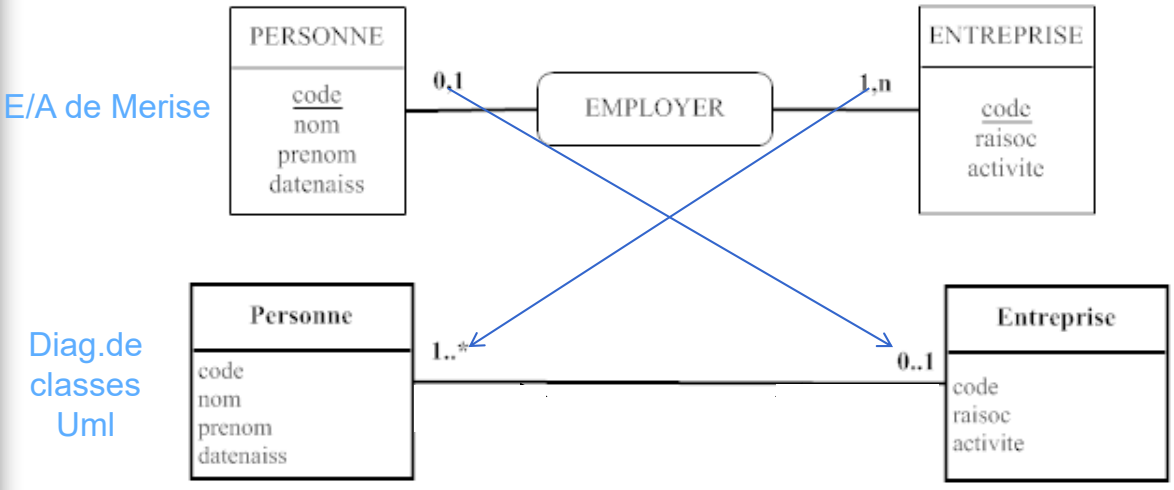
- association : nom/verbe sur ou sous le lien + rôles

- identifiant : décrit dans le dictionnaire de données

V. Le diagramme de classes UML (3)

Comparaison avec le modèle Entité/Association de Merise

Inversion position des multiplicités pour une association binaire

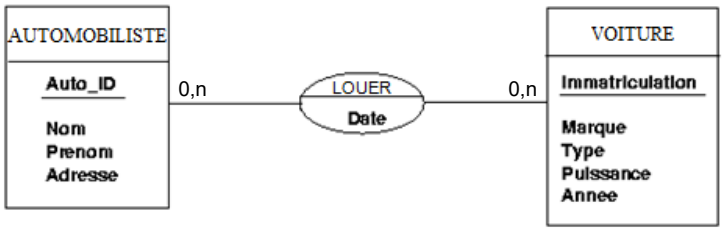


V. Le diagramme de classes UML (4)

Comparaison avec le modèle Entité/Association de Merise

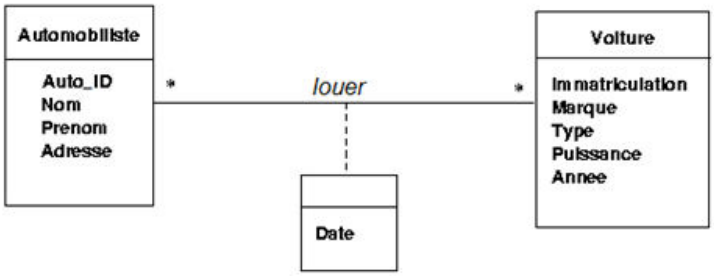
Classe d'association binaire avec propriétés

E/A de Merise



⇔ Classe-association binaire avec propriétés

Diag.de classes Uml

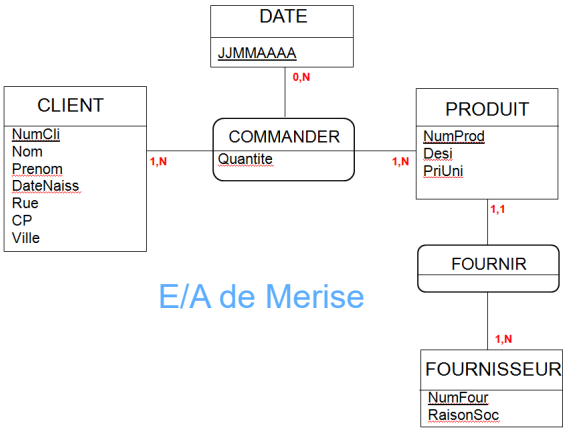


- Nom de l'association placé sur le lien entre les deux classes
- Propriétés de l'association placées dans un rectangle relié par une ligne en pointillés au trait matérialisant l'association entre les deux classes.

V. Le diagramme de classes UML (5)

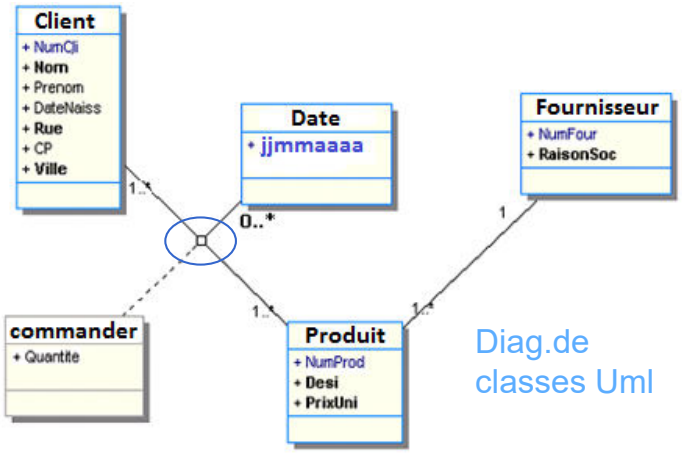
Comparaison avec le modèle Entité/Association de Merise

Classe d'association n-aire avec propriétés



E/A de Merise

⇔ Classe-association n-aire avec propriétés



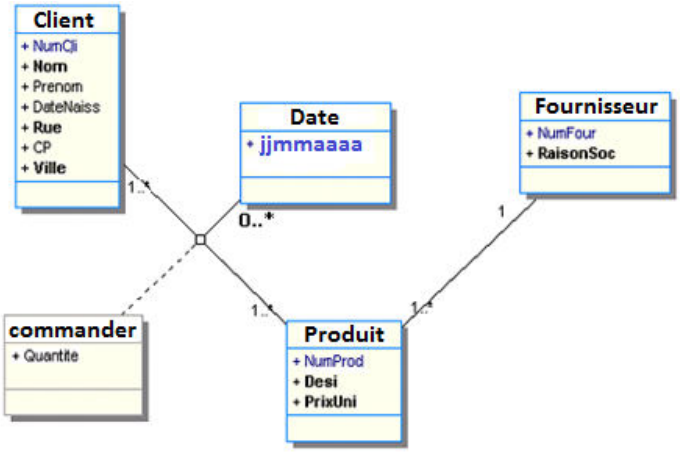
Diag.de classes Uml

- Les liens partant de chaque classe qui participe à l'association sont rattachés à un losange.
- Le nom de l'association est placé à côté du losange ou dans le premier logement du rectangle contenant les propriétés de l'association.
- Les propriétés de l'association sont placées dans un rectangle relié par une ligne en pointillés au losange.

V. Le diagramme de classes UML (6)

Modèle Conceptuel de Données version UML

- Schéma : diagramme de classes UML



Spécifications *: Les clients sont caractérisés par un numéro de client, leur nom, prénom, date de naissance, rue, code postal et ville. Ils commandent des produits à une date donnée et dans une quantité donnée.*
Les produits sont caractérisés par un numéro de produit, leur désignation et leur prix unitaire.
Chaque produit est fourni par un fournisseur unique (mais un fournisseur peut fournir plusieurs produits).
Les fournisseurs sont caractérisés par un numéro de fournisseur et leur raison sociale.

- Description textuelle (contenu identique modèle E/A de Merise)
  - Dictionnaire des données,
  - Règles de gestion,
  - Commentaires pertinents
  - …
- Tables des occurrences

Plan du cours

I. Introduction
II. Le modèle E/A de Merise : concepts de base
III. Exemple
IV. Construire un Modèle Conceptuel de Données
V. Le diagramme de classes UML : concepts de base
VI. Modélisation Logique des Données
VII. Le modèle E/A de Merise et d'UML : concepts étendus

Bibliographie Chap. VI (MLD)

- Supports de cours de collègues
- Pantxika DAGORRET
  - IUT de Bayonne et du Pays Basque, Département Informatique
- Bernard MORAND
  - IUT de Caen, Département Informatique

Plan Chap. VI (MLD)

- Niveaux d'abstraction de la modélisation
- Niveau logique – Concepts et Exemple
- Opérations de manipulation de données
- Règles de transformation
- Conclusion

Niveaux d'abstraction de la modélisation

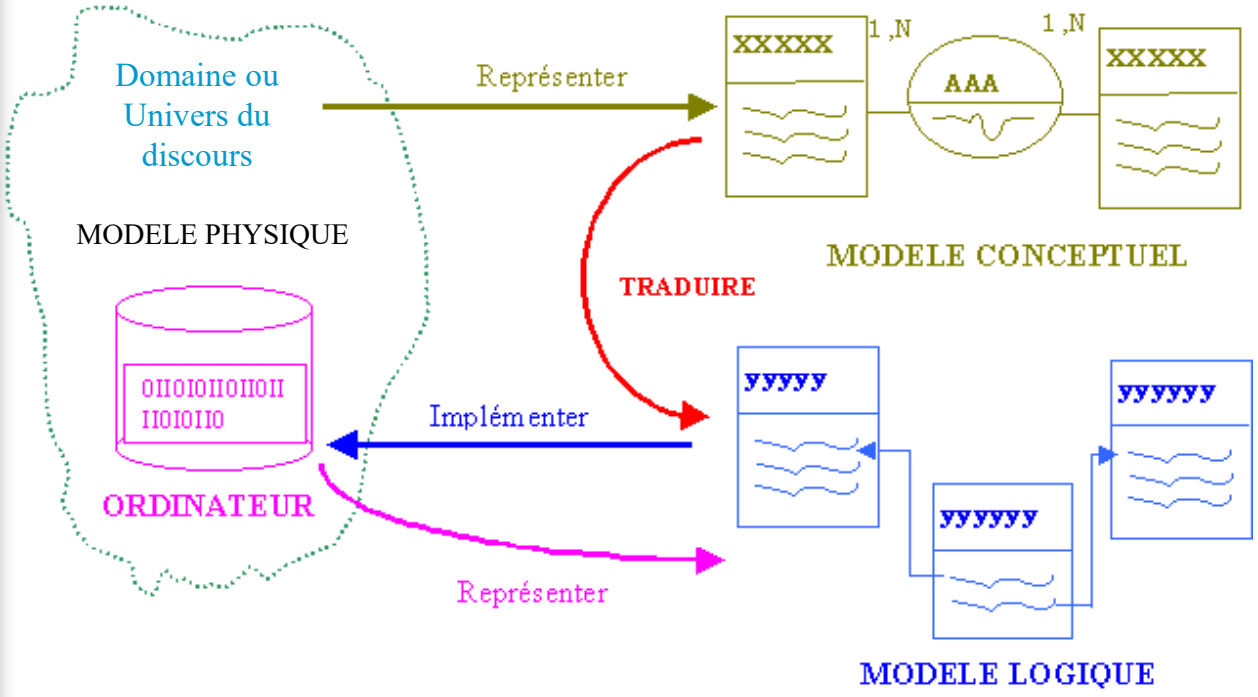
- Un de données (MCD MERISE ou Diagramme de Classes UML) est indépendant de l'état de l'art technologique.
- Il existe différentes sortes de SGBD qui ont chacun leur propre (dans l'ordre historique) :
  - Systèmes de Gestion de Fichiers (SGF) qui ne sont pas vraiment des SGBD.
  - SGBD hiérarchiques organisés selon une arborescence.
  - SGBD réseau encore appelés CODASYL (du nom de la norme à laquelle ils se réfèrent).
  - : Oracle, Access, MySQL, PostgreSQL, DB2, etc.
  - SGBDOO (Orientés Objet) : O2, Jasmine, Versant, DB40, etc. *Certains SGBDR disposent aussi d'une extension objet.*
  - SGBD XML : Tamino, eXist, etc. *Certains SGBDR disposent aussi d'une extension XML.*
- On ne sait pas implémenter aisément un modèle « conceptuel » dans une machine :
  - Une association de dimension supérieure à 2 doit être transformée.
  - Une association porteuse de données n'est pas toujours implémentable telle quelle.

Niveaux d'abstraction de la modélisation

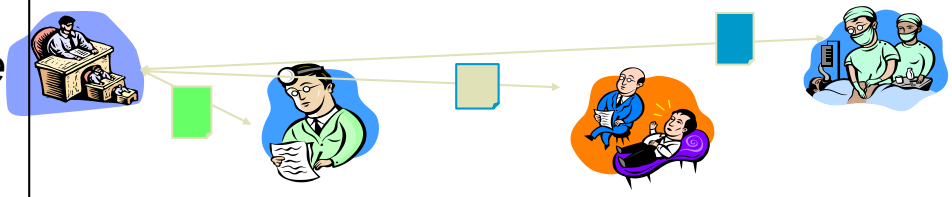
- Le est une du système tel qu'il sera implanté.
- Vu qu'il s'agit d'une représentation, on utilisera également un modèle !
- Toutefois :
  - Il ne faut pas confondre le modèle « conceptuel » (entité - association par exemple) avec le modèle « logique » (relationnel par exemple).
  - Il ne faut pas confondre le modèle « logique » (relationnel par exemple) avec son implémentation « physique » en machine (avec Access, MySQL, PostgreSQL ou Oracle par exemple).

Niveaux d'abstraction de la modélisation

- On passe du modèle conceptuel au modèle logique par une opération de



Niveaux d'abstraction de la modélisation

Réel (*monde physique*)					
Modèle conceptuel	❑ Indépendant du SGBD				
Modèle logique	❑ Dépendant du modèle conceptuel ❑ Indépendant du SGBD	Codasyl	**Relationnel** *(textuel ou graphique)*	Objet	XML
Modèle physique	❑ Dépendant du SGBD	❑ Organisation physique des données ❑ Structures de stockage des données ❑ Structures accélératrices (index)			

Niveau logique – Concepts

- Une est une partie finie du produit cartésien de n ensembles (ou **domaines**).
- Un prend ses valeurs dans un (soit l'ensemble de ses valeurs possibles). Ainsi, une est le résultat d'une opération sur les ensembles constitués par les valeurs des attributs.
- de relation : un attribut particulier (ou un ensemble d'attributs) tel qu'il ne peut exister qu'une seule valeur de cet attribut pour tous les n-uplets de la table. Ce pourrait être le cas (dans le transparent suivant) de l'attribut « code_client » (qui ressemble à notre définition d'identifiant au niveau conceptuel).
- Un est un enregistrement (une ligne) dans la base de données.
- de relation : le nombre de ses n-uplets (ou lignes).
- (ou encore) de relation : le nombre des attributs (ou colonnes). Rien à voir donc avec la (nombre d'entités reliées) d'une association.

Niveau logique – Exemple

- Exemple de TABLE (RELATION) « CLIENT » :

« relation en extension »

code	nom	adresse	ville
AXV508	Dupont	15, rue du Canada	Caen
BWY405	Durand	6, rue Méchin	Cherbourg
KAC567	Duval	25, rue Blanchi	Alençon

4 attributs

3 n-uplets ou tuples

Valeurs d'attributs

- Schéma (*textuel*) d'une relation (« *relation en intention* ») :

CLIENT (<u>code</u>, nom, adresse, ville)

Nom de relation *Clé primaire de relation* *Attributs*

Opérations de manipulation de données

- Langage SQL (*Structured Query Language*)
- *Cf.* enseignements de Manipulation de Bases de Données
  - Sélection
  - Projection
  - Jointure
  - Union
  - Intersection
  - Différence
  - Etc.

Règles de passage du Modèle Conceptuel (formalisme MERISE) au Schéma Relationnel

- Règle n°1 : Toute (classe d')entité devient une relation dans laquelle :
  - les ***attributs*** traduisent les ***propriétés*** de l'entité
  - la ***clé primaire*** traduit ***l'identifiant*** de l'entité

EMPLOYE
matricule
nom
prenom
dateEmb
qualite

Identifiant

Clé primaire

EMPLOYE (matricule, nom, prenom, dateEmb, qualite)

Propriétés

Attributs

Règles de passage du Modèle Conceptuel (formalisme Diagramme de Classes UML) au Schéma Relationnel

- Règle n°1 : Toute (classe d')entité devient une relation dans laquelle :
  - les ***attributs*** traduisent les ***propriétés*** de l'entité
  - la ***clé primaire*** traduit ***l'identifiant*** de l'entité

Employe
matricule
nom
prenom
dateEmb
qualite

Identifiant

Clé primaire

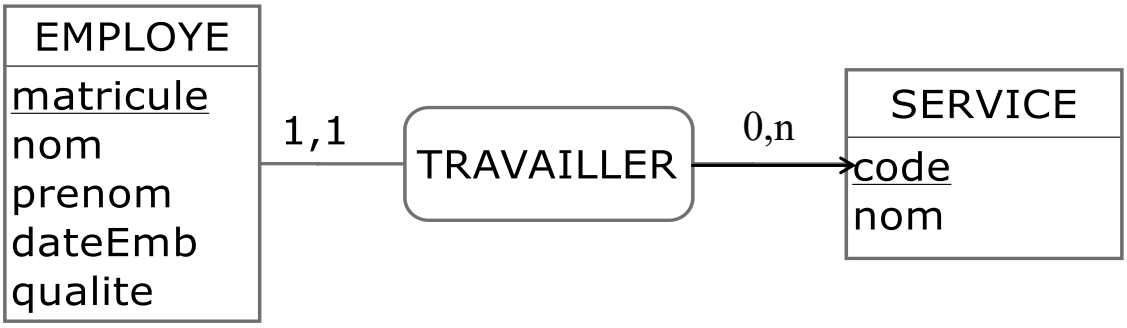
EMPLOYE (matricule, nom, prenom, dateEmb, qualite)

Propriétés

Attributs

Règles de passage du Modèle Conceptuel (formalisme MERISE) au Schéma Relationnel

- Règle n°2 : Une (classe d')association de dimension 2 avec cardinalité 1,1 (ou 0,1) se réécrit en :
  - portant dans la ***relation fille*** la clé primaire de la ***relation mère***.
  - L'**attribut** ainsi **ajouté** s'appelle ***clé étrangère***. Par convention, nous la symboliserons au moyen de **#** (*Attention, ce caractère ne fait toutefois pas partie du nom du champ dans la BD !*) et nous suffixerons avec le nom de la classe d'entité (*convention d'écriture*).

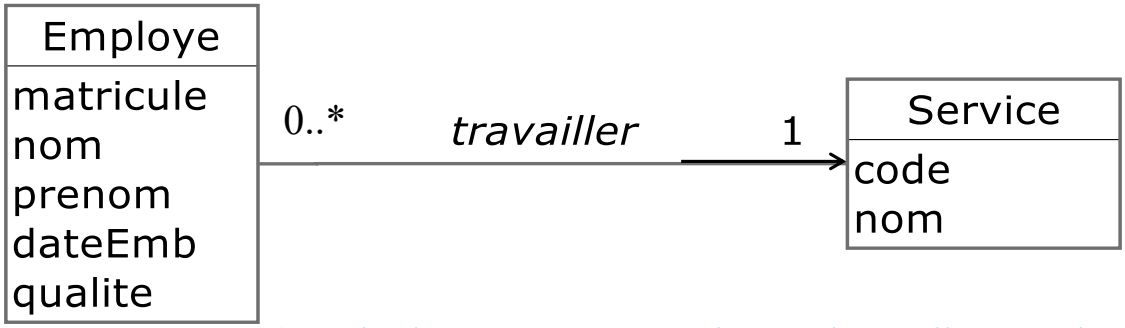


EMPLOYE (matricule, nom, prenom, dateEmb, qualite, #codeService)

SERVICE (code, nom)

Règles de passage du Modèle Conceptuel (formalisme Diagramme de Classes UML) au Schéma Relationnel

- Règle n°2 : Une (classe d')association de dimension 2 avec multiplicité 1 (ou 0..1) se réécrit en :
  - portant dans la ***relation fille*** la clé primaire de la ***relation mère***.
  - L'**attribut** ainsi **ajouté** s'appelle ***clé étrangère***. Par convention, nous la symboliserons au moyen de **#** (*Attention, ce caractère ne fait toutefois pas partie du nom du champ dans la BD !*) et nous suffixerons avec le nom de la classe d'entité (*convention d'écriture*).



EMPLOYE (matricule, nom, prenom, dateEmb, qualite, #codeService)

SERVICE (code, nom)

Règles de passage du Modèle Conceptuel au Schéma Relationnel

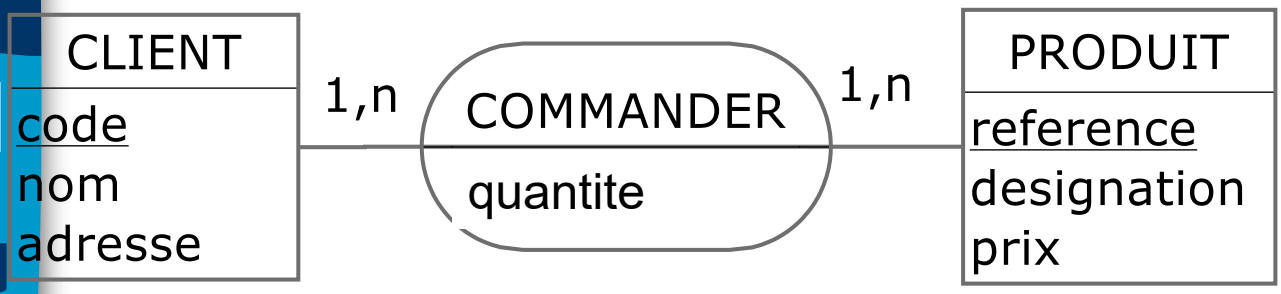
- Règle n°2 : Recette
  - On hésite quelquefois au début pour savoir dans laquelle des 2 relations, il faut porter la clé étrangère. Il faut contrôler si cet attribut est bien monovalué (*i.e.* qu'il respecte bien la 1ère Forme Normale).
  - Supposons que, par erreur, nous portions « matricule » dans SERVICE comme clé étrangère. Si tel est le cas, alors cet attribut aurait comme valeurs tous les matricules des employés de ce service !
  - Par contre, dans la solution correcte de l'exemple précédent, on voit clairement que **#codeService** ne peut posséder qu'une valeur, celle du service dans lequel travaille cet employé (il n'y en a qu'un).

Règles de passage du Modèle Conceptuel au Schéma Relationnel

- Cas particulier de la cardinalité 0,1 / multiplicité 0..1
  - La même règle s'applique mais la clé étrangère pourra avoir une « valeur nulle ». Ainsi, si un employé peut ne pas travailler dans un service, alors il se peut que **#codeService** n'ait pas de valeur. Il en résulte que l'on ne peut plus, dans ce cas, utiliser aisément le mécanisme de vérification du respect de l'intégrité référentielle.
- Remarques
  - Nous avons mentionné qu'une classe d'association avec une cardinalité 1,1 / multiplicité 1 ne devrait pas être porteuse de propriétés. On vérifie maintenant que si cela a toutefois été le cas au niveau conceptuel, alors cette propriété devient attribut de la relation enfant.
  - Attention aussi aux classes d'association ayant un minimum de 1 pour toutes les cardinalités/multiplicités (dimension n:m). Cela peut poser des contraintes lors du (dé)peuplement et nécessiter des outils de gestion évolués (triggers par exemple).

Règles de passage du Modèle Conceptuel (formalisme MERISE) au Schéma Relationnel

- Règle n°3 : Une (classe d')association de dimension 2 avec des cardinalités 1,n et-ou 0,n se réécrit en :
  - créant une relation particulière qui contient comme attributs les identifiants des 2 (classes d')entités associées
    - ces attributs constituent à eux 2 la clé primaire de la relation
    - ils sont individuellement clés étrangères
  - ajoutant la ou les éventuelles propriétés de l'association comme attributs de cette relation.

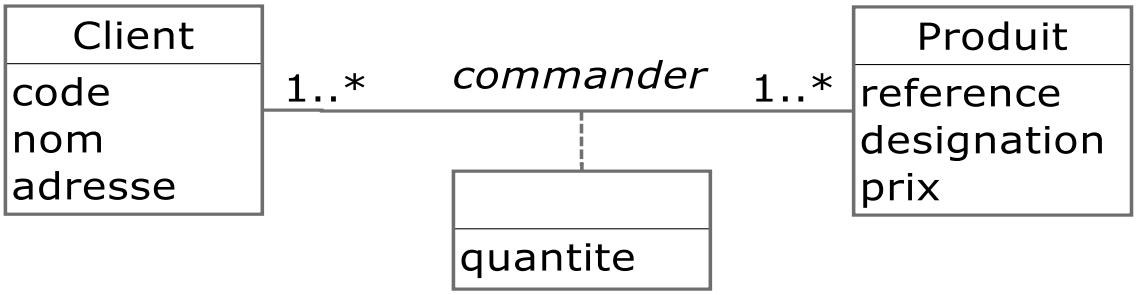


CLIENT (code, nom, adresse) **PRODUIT** (reference, designation, prix)

COMMANDER (#codeClient, #refProduit, quantite)

Règles de passage du Modèle Conceptuel (formalisme Diagramme de Classes UML) au Schéma Relationnel

- Règle n°3 : Une (classe d')association de dimension 2 avec des multiplicités 1..* et-ou 0..* se réécrit en :
  - créant une relation particulière qui contient comme attributs les identifiants des 2 (classes d')entités associées
    - ces attributs constituent à eux 2 la clé primaire de la relation
    - ils sont individuellement clés étrangères
  - ajoutant la ou les éventuelles propriétés de l'association comme attributs de cette relation.

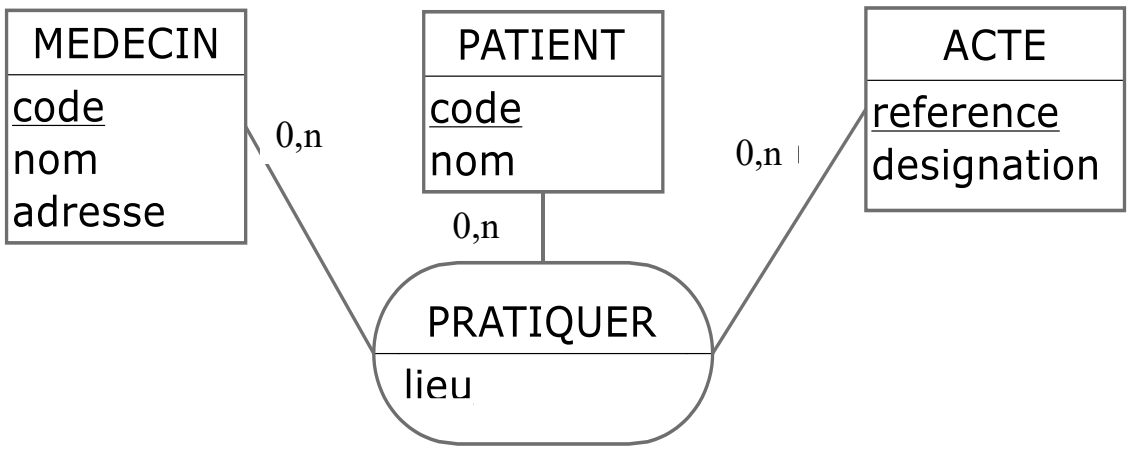


CLIENT (code, nom, adresse) **PRODUIT** (reference, designation, prix)

COMMANDER (#codeClient, #refProduit, quantite)

Règles de passage du Modèle Conceptuel (formalisme MERISE) au Schéma Relationnel

- Règle n°4 : Une (classe d')association de dimension supérieure à 2 se réécrit selon la règle n°3.



MEDECIN (code, nom, adresse)

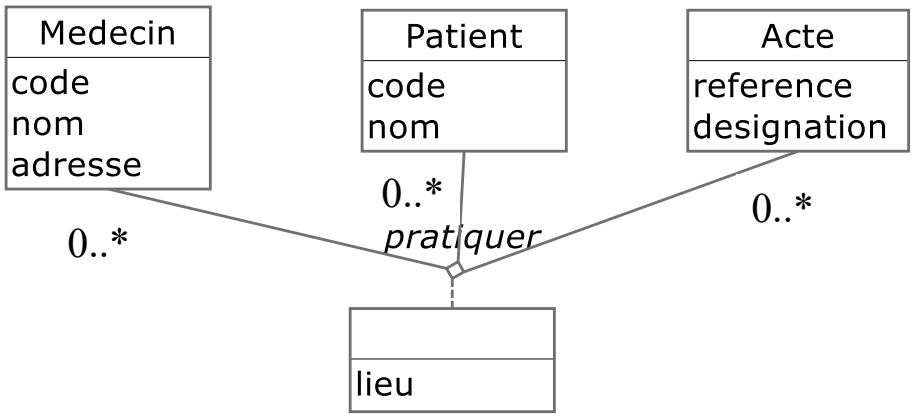
PATIENT (code, nom)

ACTE (reference, designation)

PRATIQUER (#codeMedecin, #codePatient, #refActe, lieu)

Règles de passage du Modèle Conceptuel (formalisme Diagramme de Classes UML) au Schéma Relationnel

- Règle n°4 : Une (classe d')association de dimension supérieure à 2 se réécrit selon la règle n°3.



MEDECIN (code, nom, adresse)
PATIENT (code, nom)
ACTE (reference, designation)
PRATIQUER (#codeMedecin, #codePatient, #refActe, lieu)

Conclusion

- Le schéma relationnel peut se présenter sous **forme textuelle**, comme dans ce document, en respectant des normes/conventions d'écriture (exemple : nom de table en majuscule, clé primaire soulignée, clé étrangère précédée de #) mais aussi sous **forme graphique** (comme dans MS Access par exemple).
- **Ne pas oublier de mettre à jour le dictionnaire des données** en :
  - ajoutant les attributs qui sont les clés étrangères.
  - ajoutant/renseignant une colonne pour le type physique (en précisant le SGBD choisi/imposé)

- En conclusion, les règles de passage d'un modèle de niveau conceptuel en niveau logique puis physique sont totalement formalisables. Par conséquent, on peut les automatiser au moyen d'un
  - Un AGL vous aide ainsi à dessiner votre modèle E-A et génère ensuite automatiquement le schéma de la BD ; cela pouvant fonctionner dans les 2 sens (rétro-ingénierie).
  - Exemples d'AGL : WinDev de PCSoft, AMCDesignor de Sybase, Rational Rose d'IBM, Modelio, BOUML, ArgoUML, **MySQL Workbench** (ex DBDesigner), **Looping** (https://www.looping-mcd.fr/), etc.

Conclusion

- Le **Dictionnaire des Données** (**mis à jour**) est aussi utile pour estimer le **calcul des volumes de données** nécessaires à la mise en place de la base de données relationnelle grâce à une estimation du nombre d'enregistrements de chacune des tables. Ceci est particulièrement pertinent en informatique embarquée ou pour les gros volumes.
- Exemple pour la table EMPLOYE :

Libellé	Signification	Domaine			Contraintes	Taille	Exemple
		niveau conceptuel	niveau logique	niveau physique (Oracle)			
code	Code identifiant chaque employé	numérique	entier	NUMBER	Clé primaire	5	12345
nom	Patronyme de chaque employé	alphabétique	chaîne de caractères	VARCHAR2		25	Kükürütx
codeService	Code du service pour lequel travaille chaque employé	alphanumér.	chaîne de caractères	VARCHAR2	Clé étrangère faisant référence au code unique du Service. Obligatoirement renseigné.	6	info02
…							

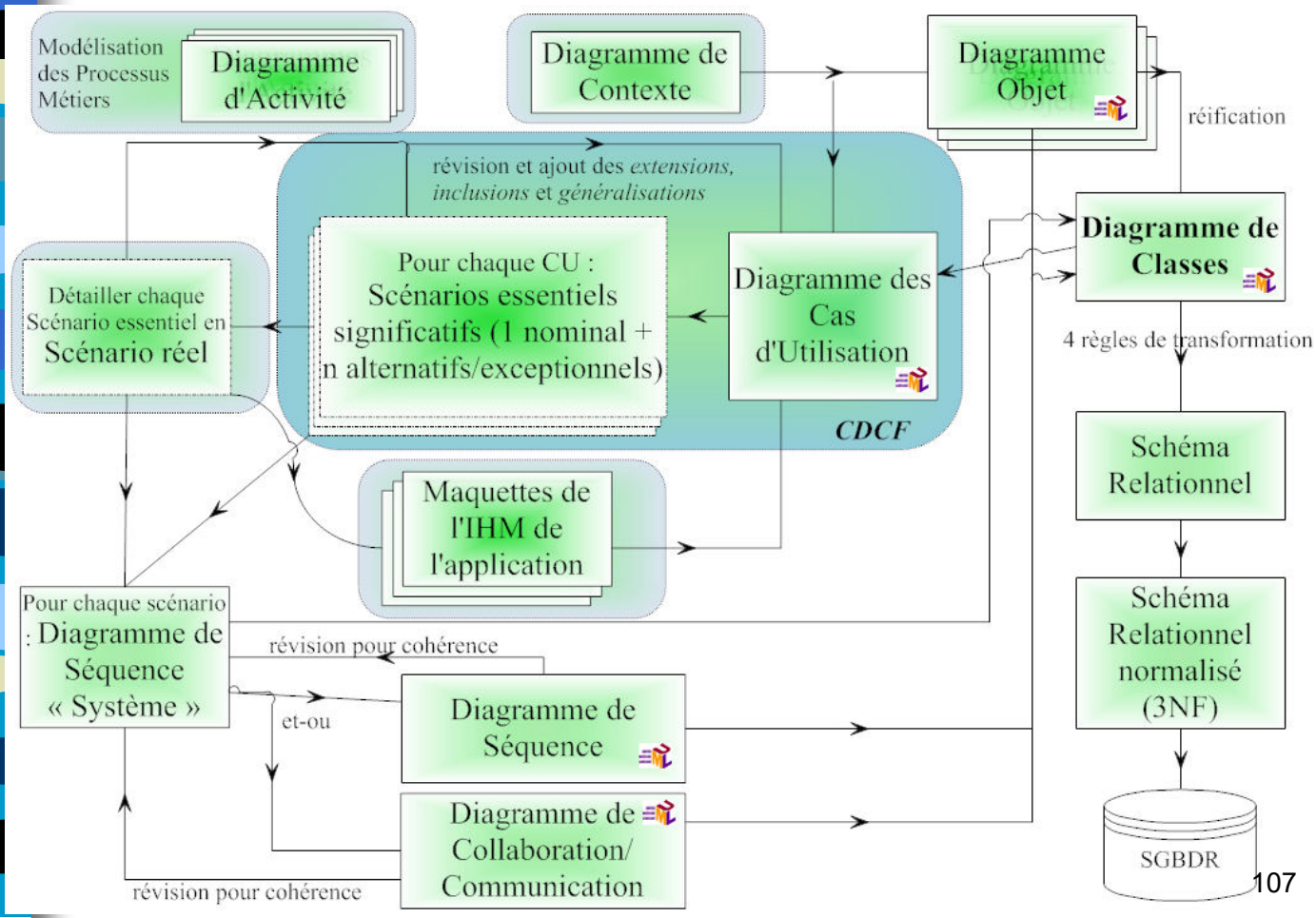
- Si on suppose que un entier est codé sur un octet, un caractère sur un octet, chaque enregistrement de employé sera codé sur 36 octets. Si on suppose que notre SI traitera 1000 employés, notre SI nécessitera au minimum 36 Ko et ainsi de suite..

Conclusion

- Quelques remarques de vocabulaire.
- Le « *Domaine* » est encore appelé « *Univers du discours* ».
- On parle aujourd'hui « *d'objets métiers* » ou encore « *d'ingénierie des besoins* ».
- Vous devez ici faire « naturellement » le lien avec vos cours de (manipulation de) Bases de Données. Au niveau physique, le vocabulaire dépend grandement du SGBDR choisi.

Niveau conceptuel	Niveau logique	Niveau physique
Classe d'entité		
Classe d'association		
Propriété		
Identifiant		
Occurrence		

Notre processus de développement simplifié



Plan du cours

I. Introduction
II. Le modèle E/A de Merise : concepts de base
III. Exemple
IV. Construire un Modèle Conceptuel de Données
V. Le diagramme de classes UML : concepts de base
VI. Modélisation Logique des Données
VII. Le modèle E/A de Merise et d'UML : concepts étendus

VII. Extensions du modèle E/A de Merise et d'UML

Certaines contraintes ne sont pas représentables par le seul formalisme de base (classe d'entité, classe d'association, propriétés, multiplicités/cardinalités...) mais correspondent à des règles que doit satisfaire le modèle pour être fidèle et cohérent avec l'activité à représenter.

=> ces contraintes seront représentées sous forme de **commentaires** *associés au* ou *dans* le dictionnaire de données

et parfois,

=> des **extensions** aux concepts de base sont proposées par le modèle E/A étendu de Merise et le diagramme de classes UML.

VII. Extensions du modèle E/A de Merise et d'UML

Extensions proposées

E/A de Merise

- Généralisation/Spécialisation de classes d'entité
- Contraintes entre les types/classes d'association : contraintes ensemblistes, CIF
- *Entité complexe*

Diagramme de classes d'UML

- Contraintes, OCL, Notes
- Généralisation/Spécialisation/Héritage
- Association de type agrégation, composition
- *Association réifiée*

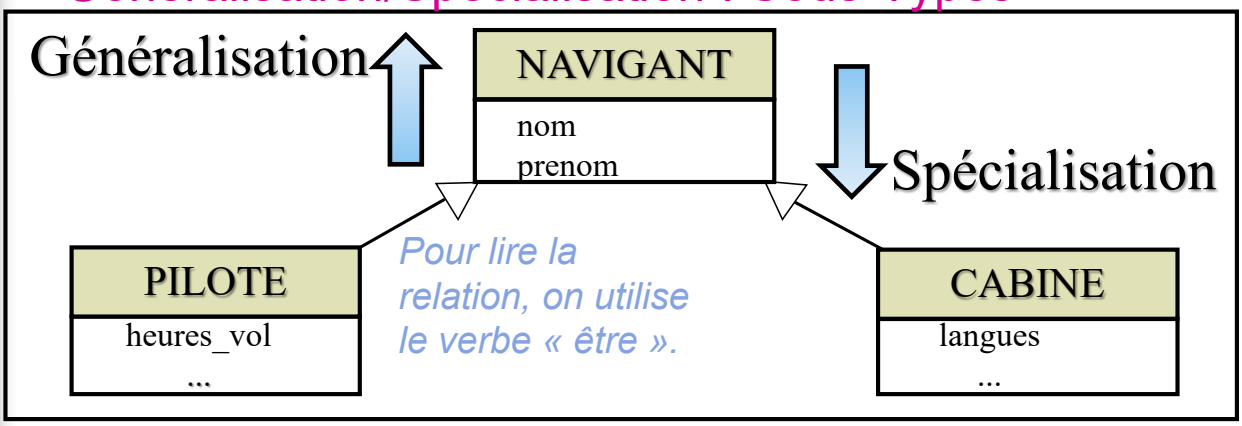
VII.1 Extensions du modèle E/A de Merise – Gen/Spe (1)

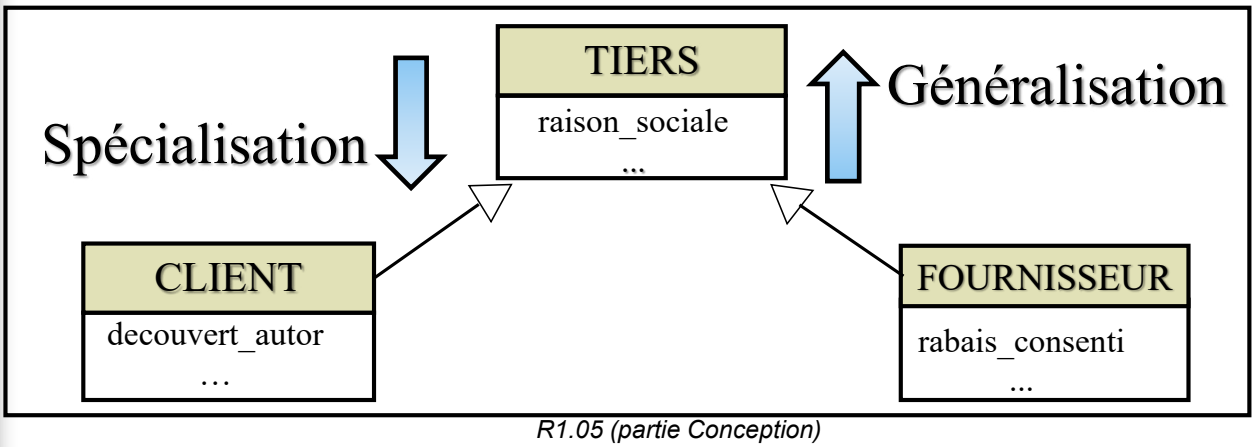
Généralisation/Spécialisation

- Cas où des types d'entité partagent des caractéristiques communes
  - ***spécialisation*** : création de classes d'entité nouvelles, rattachées à une classe d'entité plus large, avec des propriétés spécifiques ou supplémentaires (=> le type d'entité le plus général est d'abord caractérisé). ***conception descendante***
  - ***généralisation*** : regroupement de caractéristiques communes à plusieurs classes d'entité au sein d'une nouvelle classe d'entité (=>les types/classes d'entité les plus particuliers sont d'abord caractérisés. ***conception ascendante***
  - en pratique les deux processus sont souvent combinés.

VII.1 Extensions du modèle E/A de Merise – Gen/Spe (2)

Généralisation/Spécialisation : Sous-Types





VII.1 Extensions du modèle E/A de Merise – Gen/Spe (3)

Généralisation/Spécialisation : Sous-Types

Exemple : un tiers peut être à la fois fournisseur et client.

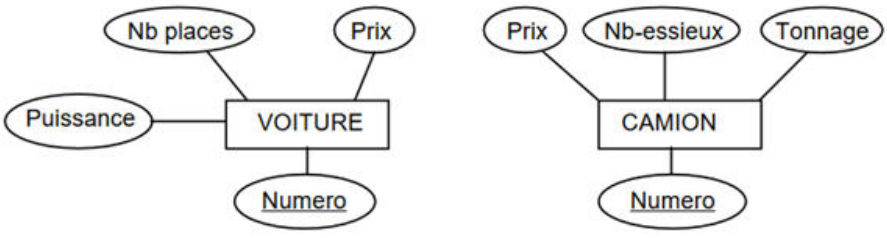
La spécialisation consiste à :

- modéliser une classe d'entité TIERS, décrite par les caractéristiques communes aux clients et aux fournisseurs;
- créer deux classes d'entité CLIENT et FOURNISSEUR comme 2 spécialisations particulières de la classe d'entité TIERS, avec des propriétés supplémentaires, et éventuellement des participations particulières à des associations.

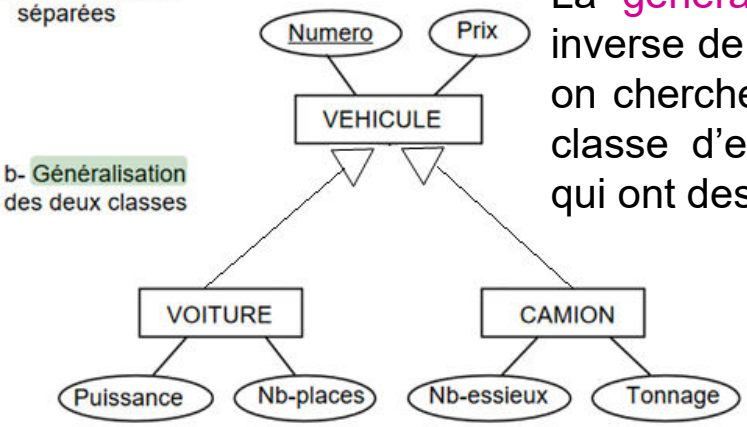
CLIENT et FOURNISSEUR sont alors sous-types ou sous-classes de la classe d'entité TIERS.

VII.1 Extensions du modèle E/A de Merise – Gen/Spe (4)

Généralisation/Spécialisation : Sous-Types



a- Deux classes séparées



b- Généralisation des deux classes

La généralisation est le processus inverse de celui de la spécialisation : on cherche à rattacher à une même classe d'entité des classes d'entité qui ont des propriétés communes.

Généralisation de deux classes

VII.1 Extensions du modèle E/A de Merise – Gen/Spe (5)

Généralisation/Spécialisation : Héritage

Une occurrence d'une sous-classe d'entité est caractérisée par toutes les propriétés de la super-classe. On dit qu'elle hérite des propriétés de la super-classe. Elle hérite aussi des associations auxquelles participe la super-classe.

En plus des propriétés héritées, une occurrence d'une sous-classe peut posséder des propriétés spécifiques et intervenir dans des associations spécifiques.

Le symbole utilisé pour la relation d'héritage ou de généralisation/spécialisation est une *flèche avec un trait plein dont la pointe est un triangle fermé*.

VII.2 Extensions du modèle E/A de Merise - Contraintes (1)

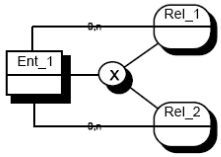
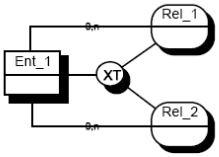
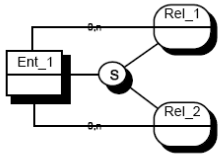
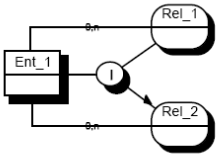
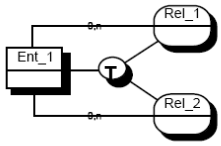
Contraintes inter-associations : contraintes ensemblistes

Elles sont des **conditions de coexistence** d'occurrences d'association :

- contraintes d'*exclusion*,
- contraintes de *simultanéité*,
- contraintes de *totalité*,
- contraintes d'*exclusion* et *totalité*,
- contraintes d'*inclusion*.

VII.2 Extensions du modèle E/A de Merise - Contraintes (2)

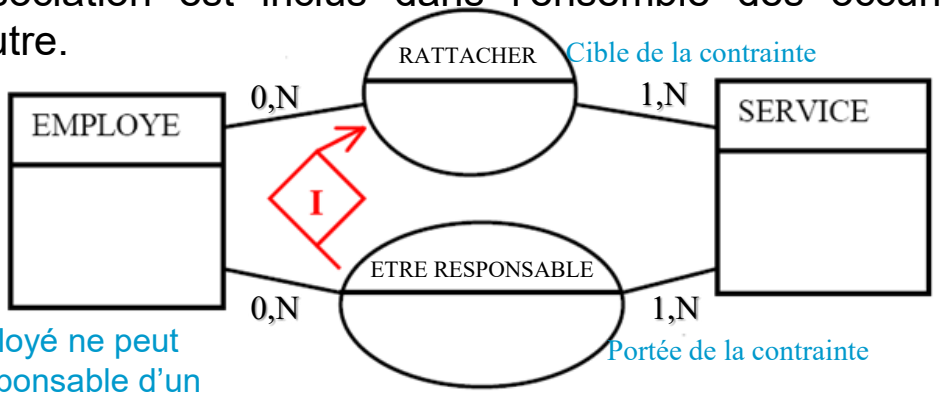
Contraintes inter-associations : contraintes ensemblistes

	**EXCLUSION** Si une occurrence de l'entité Ent_1 participe à la relation Rel_1, elle ne peut pas participer à la relation Rel_2 et réciproquement (possibilité d'orientation de cette exclusion)		**EXCLUSION et TOTALITÉ** Toute occurrence de l'entité Ent_1 participe au moins soit à la relation Rel_1, soit à la relation Rel_2, mais pas aux deux à la fois.
	**SIMULTANÉITÉ** Toute occurrence de l'entité Ent_1 participant à la relation Rel_1 participe simultanément à la relation Rel_2.		**INCLUSION** Si une occurrence de l'entité Ent_1 participe à la relation Rel_1, elle participe à la relation Rel_2 (mais pas réciproquement).
	**TOTALITÉ** Toute occurrence de l'entité Ent_1 participe au moins à l'une des deux relations Rel_1 ou Rel_2.		

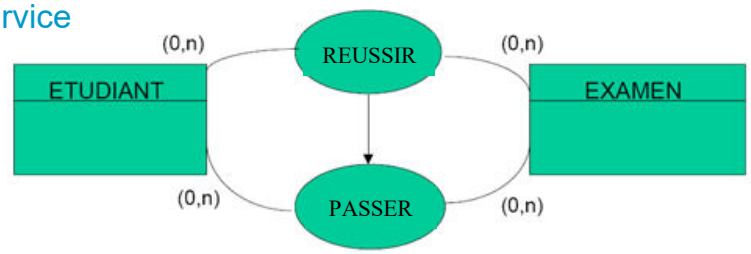
VII.2 Extensions du modèle E/A de Merise - Contraintes (3)

Contraintes inter-associations ensemblistes : inclusion

- Permet d'exprimer que l'ensemble des occurrences d'une association est inclus dans l'ensemble des occurrences de l'autre.



Un employé ne peut être responsable d'un service s'il n'est pas rattaché à ce service

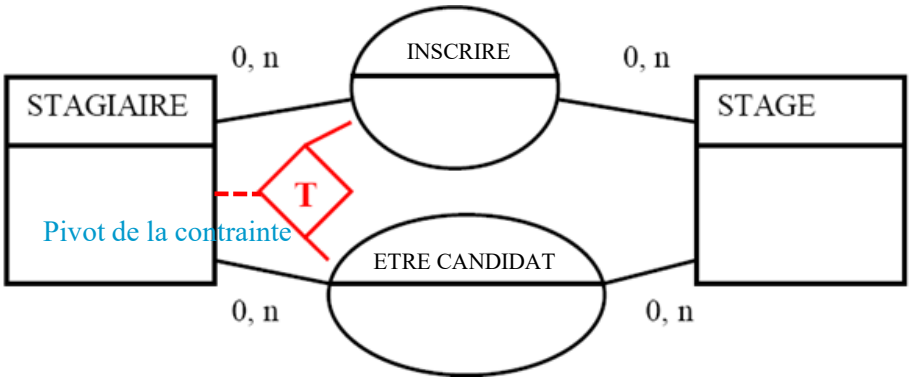


Le pivot de la contrainte est composé des entités communes à la portée et la cible.

VII.2 Extensions du modèle E/A de Merise - Contraintes (4)

Contraintes inter-associations ensemblistes : totalité (ou inclusif)

- Permet de vérifier que toutes les occurrences d'une entité impliquée dans 2 ou plusieurs associations soient présentes dans au moins l'une d'entre elles.



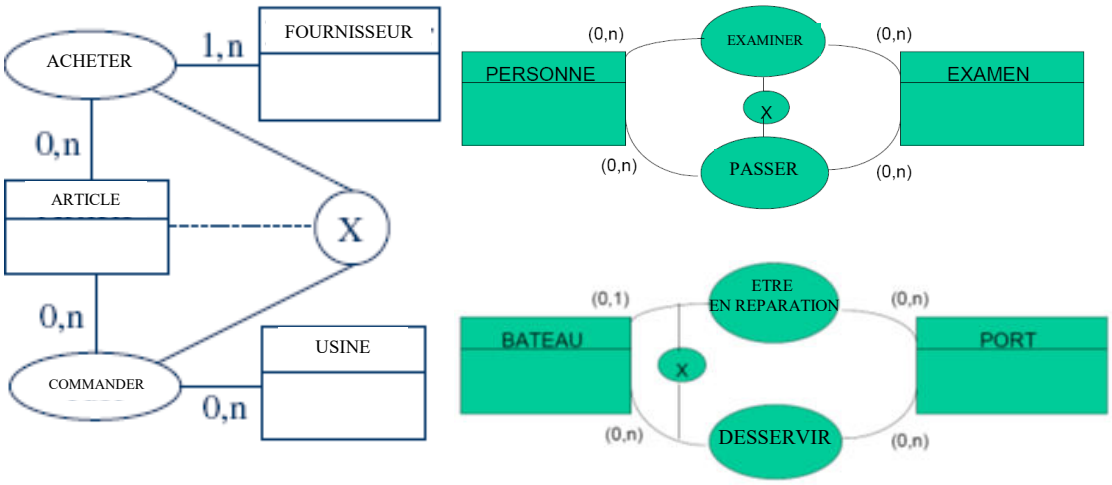
Soit un centre de formation dans lequel une personne est considérée stagiaire dès qu'elle pose sa candidature à un stage de formation.

Un stagiaire est soit :
- candidat,
- inscrit,
- les deux

VII.2 Extensions du modèle E/A de Merise - Contraintes (5)

Contraintes inter-associations ensemblistes : exclusion

- Interdit qu'une occurrence d'une entité impliquée dans deux ou plusieurs associations soit présente dans deux d'entre elles.

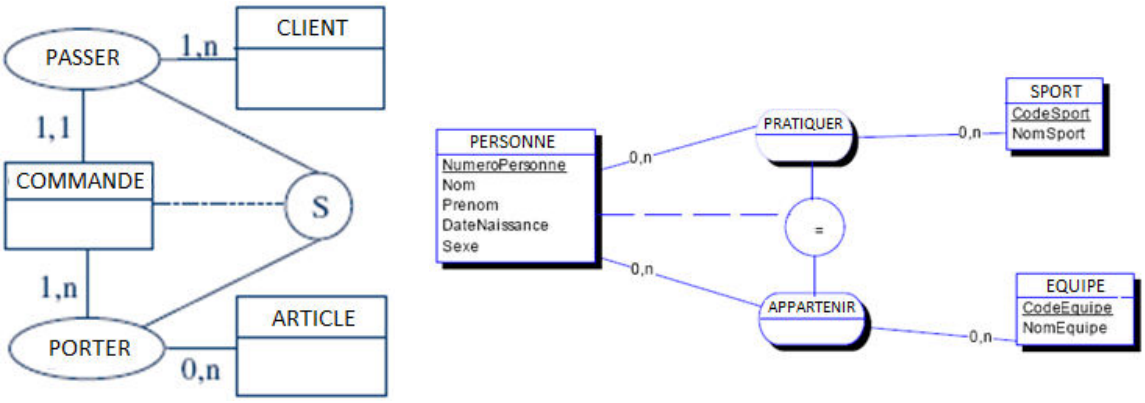


Exemple : un article peut être soit acheté à un fournisseur soit fabriqué par une usine mais pas les 2.

VII.2 Extensions du modèle E/A de Merise - Contraintes (6)

Contraintes inter-associations ensemblistes : simultanéité ou égalité

- Toute occurrence d'une entité participe de façon **simultanée** à deux ou plusieurs associations. C'est la combinaison de 2 **inclusions symétriques**.

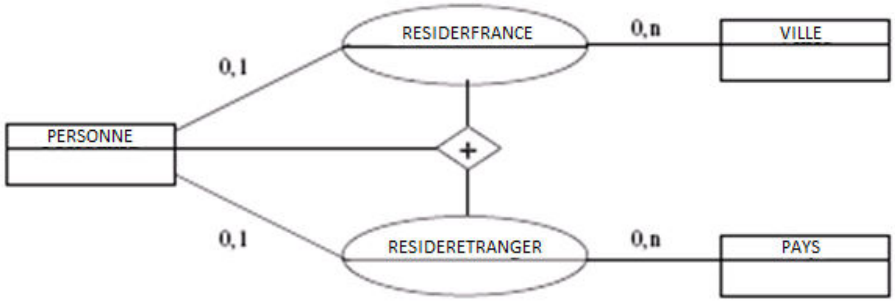


Usage du symbole S ou = pour symboliser une contrainte de simultanéité.

VII.2 Extensions du modèle E/A de Merise - Contraintes (7)

Contraintes inter-associations ensemblistes : ou exclusif

- C'est la combinaison d'une contrainte de **totalité** et d'**exclusion**, appelée aussi **contrainte de partition**



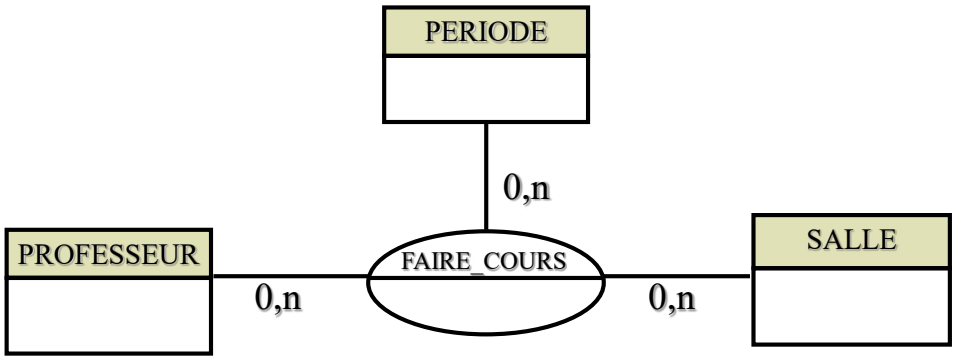
Exemple : une personne réside obligatoirement soit en France, soit à l'étranger.

Usage du symbole + ou XT pour symboliser une contrainte de totalité

VII.2 Extensions du modèle E/A de Merise - Contraintes (8)

Contrainte d'Intégrité Fonctionnelle

Exemple: *Pour une période d'emploi du temps (ex: le mercredi de 9 h à 12 h), un professeur ne fait un cours que dans une seule salle.*



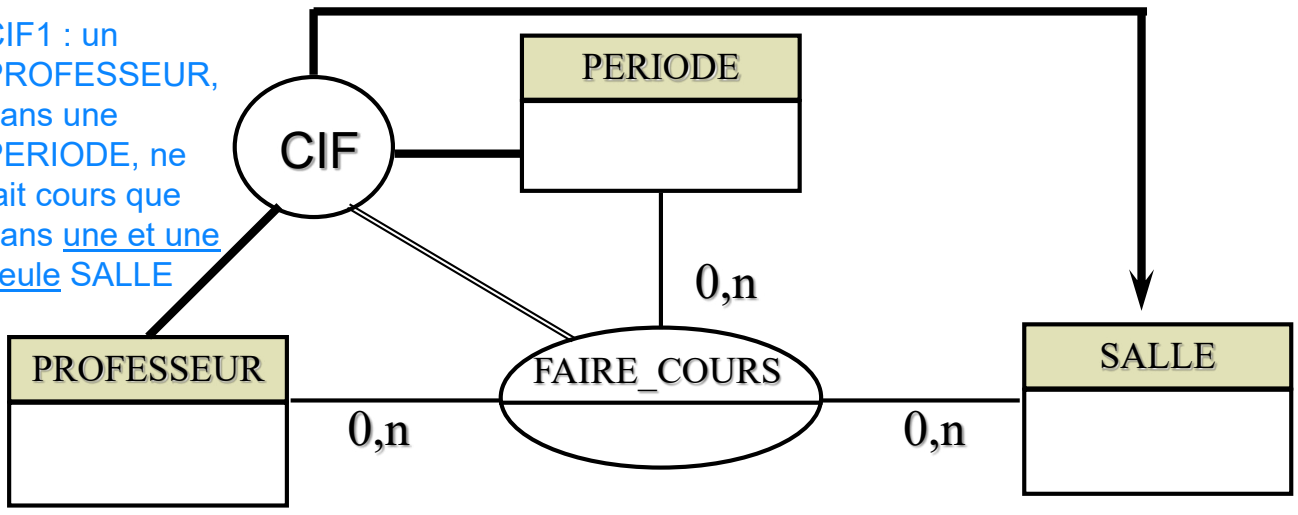
Dans le modèle E/A proposé, il est tout à fait possible d'avoir le même professeur donnant un cours dans une autre salle pour la même période.

VII.2 Extensions du modèle E/A de Merise - Contraintes (9)

Contrainte d'Intégrité Fonctionnelle

- Cas de plusieurs types d'entité avec une association type n-aire : on indique la « **CIF** » par un cercle reliant les types d'entité **sources** au type d'entité **cible** de la dépendance fonctionnelle.

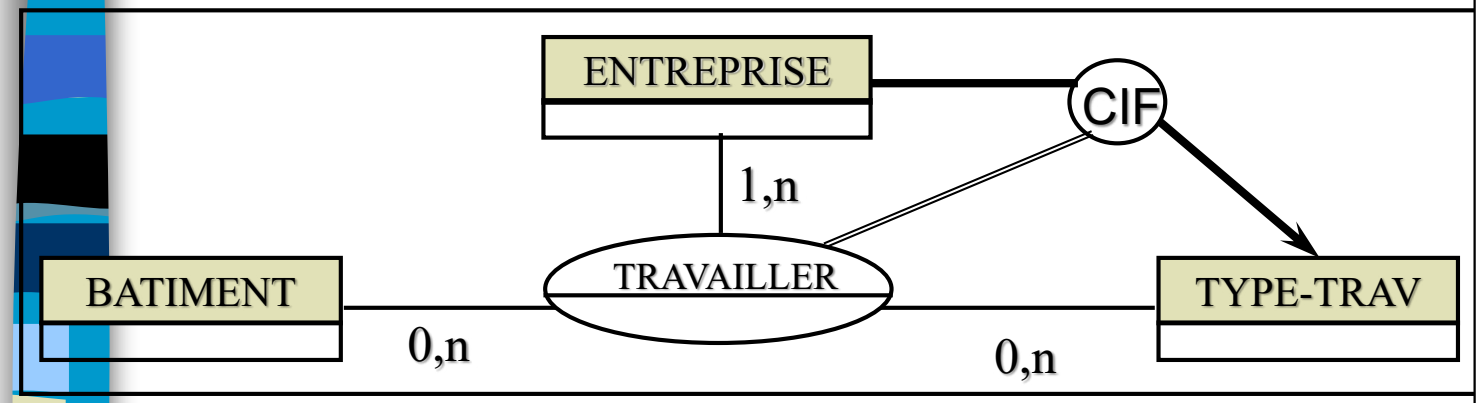
CIF1 : un PROFESSEUR, dans une PERIODE, ne fait cours que dans une et une seule SALLE



VII.2 Extensions du modèle E/A de Merise - Contraintes (10)

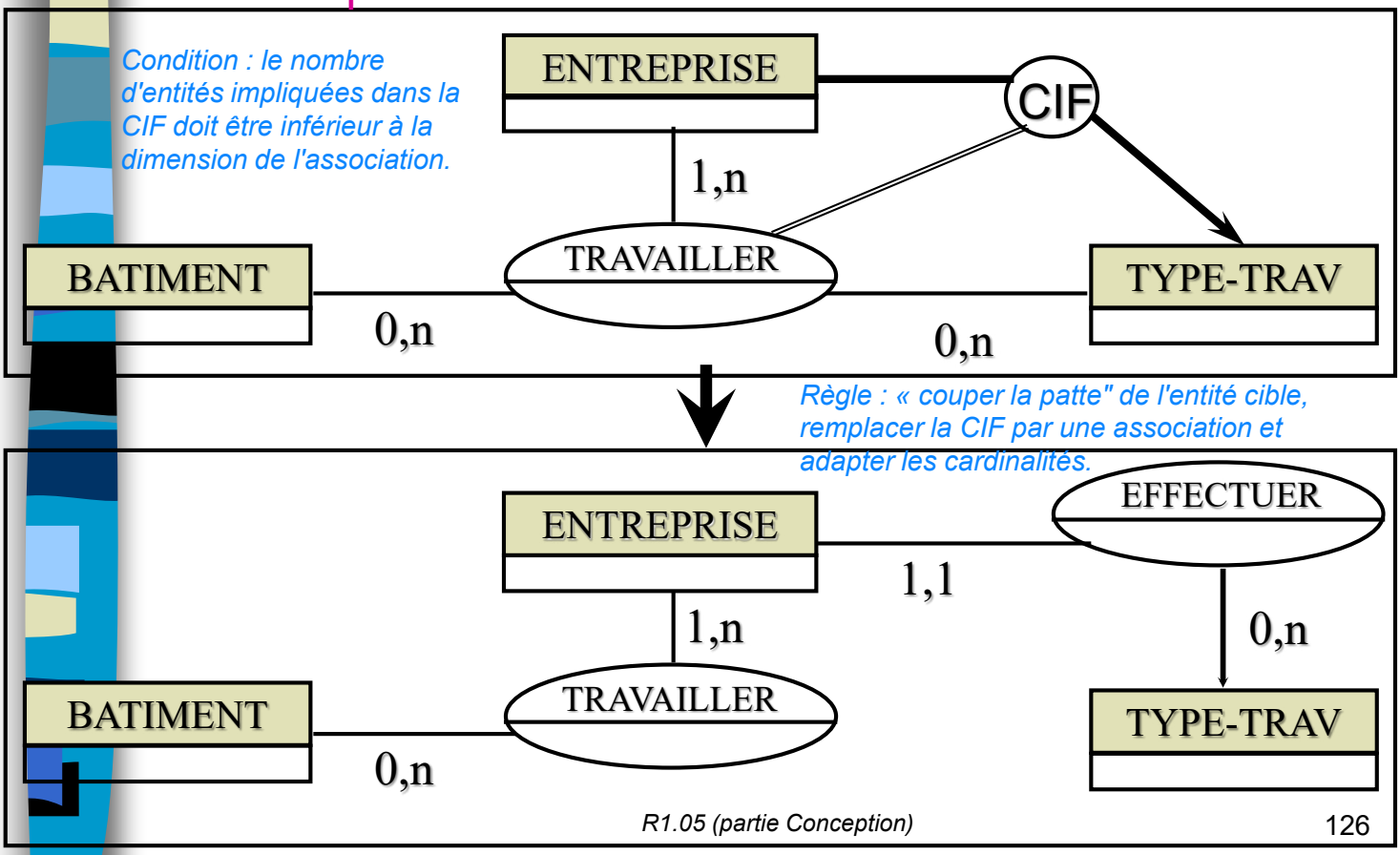
Contrainte d'Intégrité Fonctionnelle

- Cas de plusieurs types d'entité avec une association type n-aire : on indique la « **CIF** » par un cercle reliant les types d'entité **sources** au type d'entité **cible** de la dépendance fonctionnelle.



VII.2 Extensions du modèle E/A de Merise - Contraintes (11)

Contrainte d'Intégrité Fonctionnelle (CIF) => Décomposition d'une association n-aire

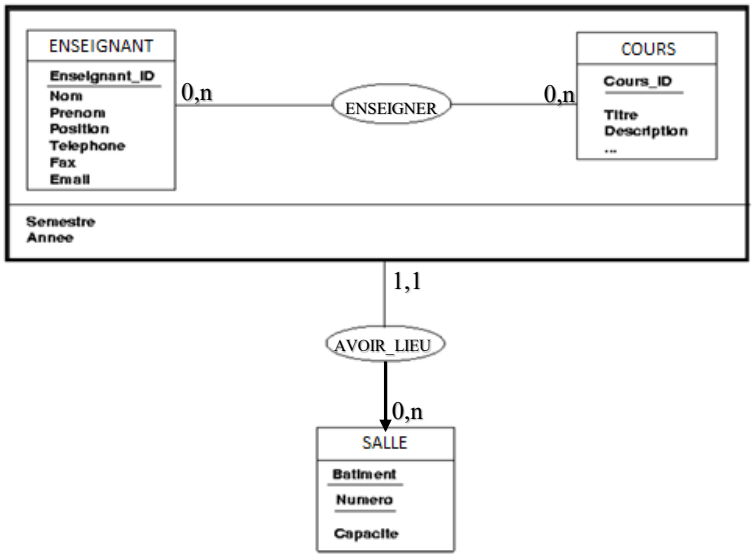


VII.2 Extensions du modèle E/A de Merise - Contraintes (12)

Contrainte d'Intégrité Fonctionnelle (CIF) => Décomposition d'une association n-aire

- Simplification d'une association n-aire objet d'une CIF

  Utiliser les **entités complexes** ou **agrégats** ou **pseudo-entités**



Agrégat ou Pseudo-entité

VII.3 Autres concepts du diagramme de classes UML - Contraintes (1)

Pour préciser la spécification…

Contraintes

- Permet de spécifier une propriété qu'un élément modélisé doit vérifier.
- Peut préciser le rôle ou la portée de tout élément d'un modèle UML: classe, attribut, opération, association...
  - Par exemple, sur une association, elles peuvent restreindre le nombre d'instances visées.
- Les contraintes peuvent s'exprimer :
  - en langage naturel (dans les notes et/ou les commentaires)
  - et/ou en **OCL** (***O**bject **C**onstraint **L**anguage*) qui est un langage mathématique formel.

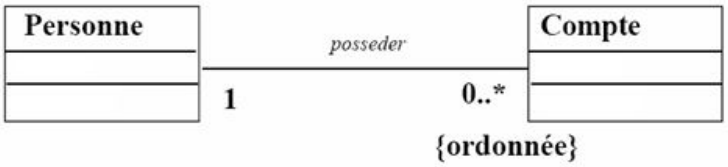
    Graphiquement, il s'agit d'un texte encadré d'accolades, positionné près de l'élément de modélisation concerné. Si la contrainte concerne deux éléments, elle est positionnée sur une ligne pointillée qui relie les deux éléments.

VII.3 Autres concepts du diagramme de classes UML - Contraintes (2)

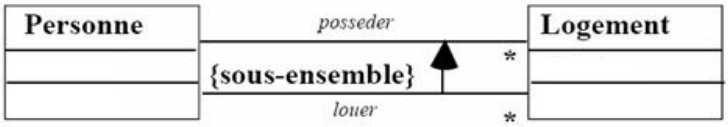
Pour préciser la spécification…

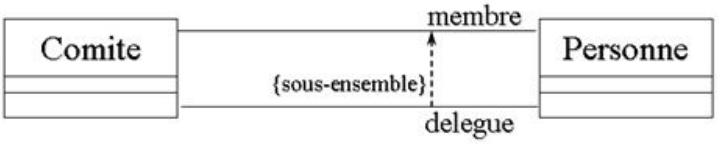
Contraintes

- contrainte `{ordonnée}` : une relation d'ordre décrit les objets placés dans la collection : l'ordre doit être maintenu,

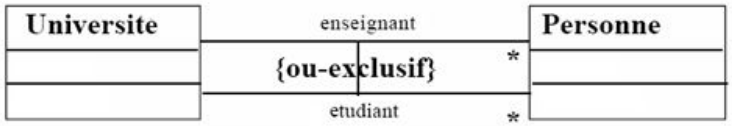


- contrainte `{sous-ensemble}` : une collection est incluse dans une autre collection,





- contrainte `{ou-exclusif}` : pour un objet donné, une seule association est valide,

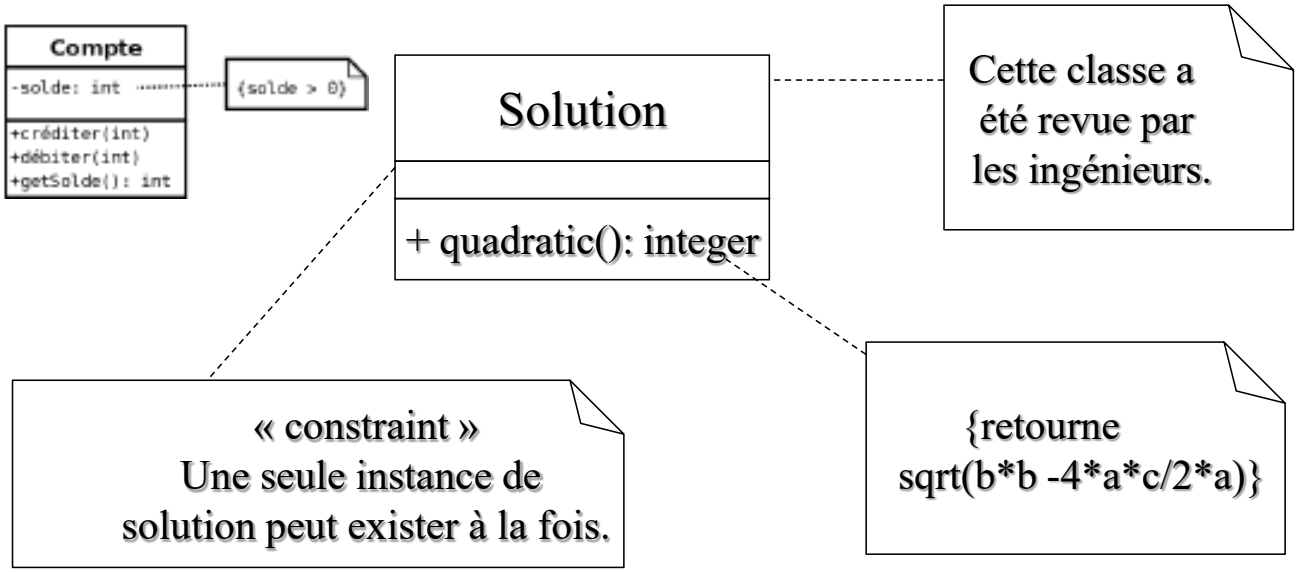


VII.3 Autres concepts du diagramme de classes UML - Contraintes (3)

Pour préciser la spécification…

Notes

- Permet d'afficher un commentaire ou toute information textuelle, telle la spécification d'une opération ou une contrainte sur la classe ou ses instances.



VII.4 Autres concepts du diagramme de classes UML - Gen/Spe (1)

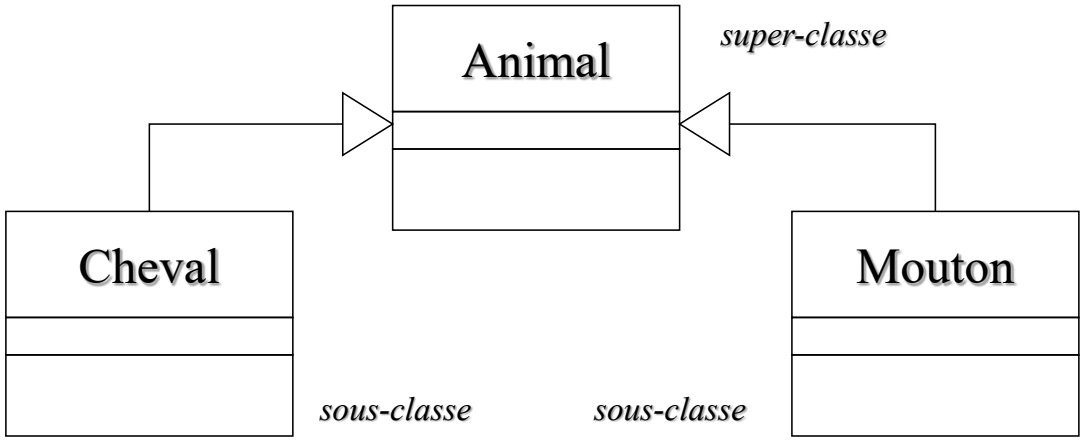
Généralisation/Spécialisation/Héritage

- La **généralisation** décrit une relation entre une classe générale (classe de base ou classe parent ou **super-classe**) et une classe spécialisée (**sous-classe**).
- La classe spécialisée est intégralement cohérente avec la classe de base, mais comporte des informations supplémentaires (attributs, opérations, associations).
- Dans le langage UML, ainsi que dans la plupart des langages objet, cette relation de généralisation se traduit par le concept **d'héritage**.

VII.4 Autres concepts du diagramme de classes UML - Gen/Spe (2)

Généralisation/Spécialisation/Héritage

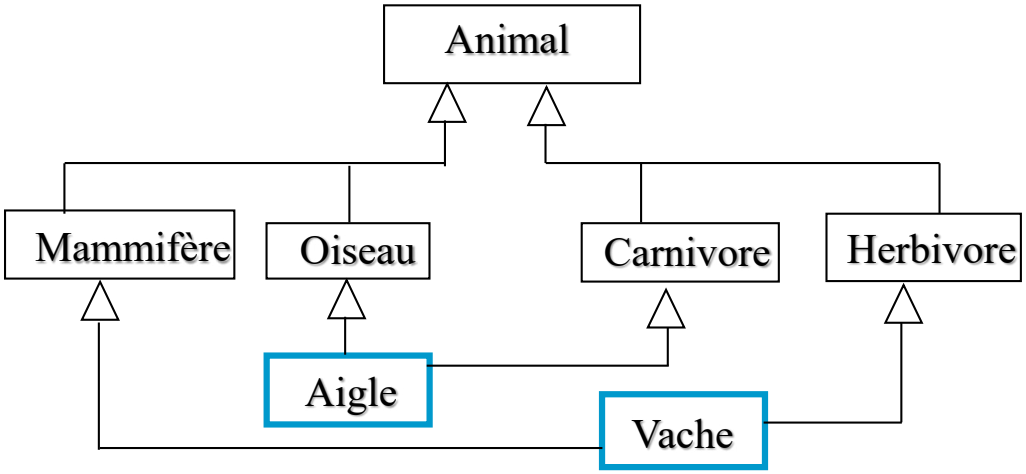
- Pour lire la relation, on utilise le verbe « être ». Le symbole utilisé pour la relation d'héritage ou de généralisation est une *flèche avec un trait plein dont la pointe est un triangle fermé* désignant le cas le plus général.



VII.4 Autres concepts du diagramme de classes UML - Gen/Spe (3)

Type d'héritage

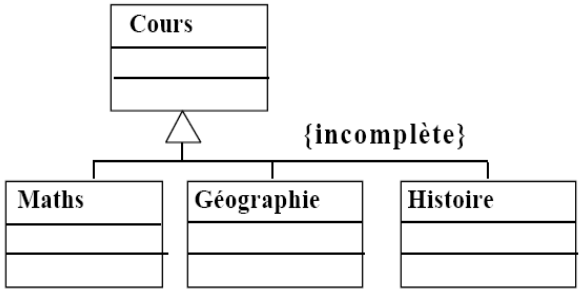
- une super-classe peut avoir plusieurs sous-classes.
- une sous-classe peut avoir plusieurs super-classes; la généralisation est alors dite **multiple**.



VII.4 Autres concepts du diagramme de classes UML - Gen/Spe (4)

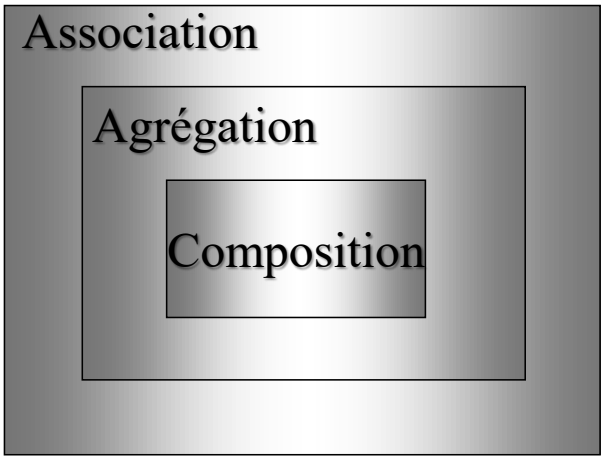
Contraintes sur hiérarchies de classes
contraintes d'intersection/disjonction

- **contrainte** `{Disjoint}` **ou** `{Exclusif}` : une sous-classe ne peut être sous-classe que d'une super-classe,
- **contrainte** `{Chevauchement}` **ou** `{Inclusif}` : une sous-classe peut être sous-classe de plusieurs super-classes (généralisation multiple possible),
- **contrainte** `{Complete}` : la généralisation est terminée, plus possible de rajouter de nouvelles sous-classes,
- **contrainte** `{Incomplete}` : il est possible de rajouter de nouvelles sous-classes :



VII.5 Autres concepts du diagramme de classes UML - Association (1)

Association, agrégation, composition



VII.5 Autres concepts du diagramme de classes UML – Association (2)

Agrégation

- Est une association particulière qui exprime une **union marquée** et une **relation de subordination** entre deux instances (un agrégat et une instance agrégée)
- Relation d'inclusion de type **« possède »** ou **« a pour partie »**
- Relation asymétrique, transitive.
- À un même moment, une instance agrégée peut être impliquée ou pas dans plus d'une association.
- Une instance agrégée peut vivre sans agrégat et inversement i.e. le cycle de vie de l'agrégat et de l'instance agrégée sont indépendants.



VII.5 Autres concepts du diagramme de classes UML - Association (3)

Composition

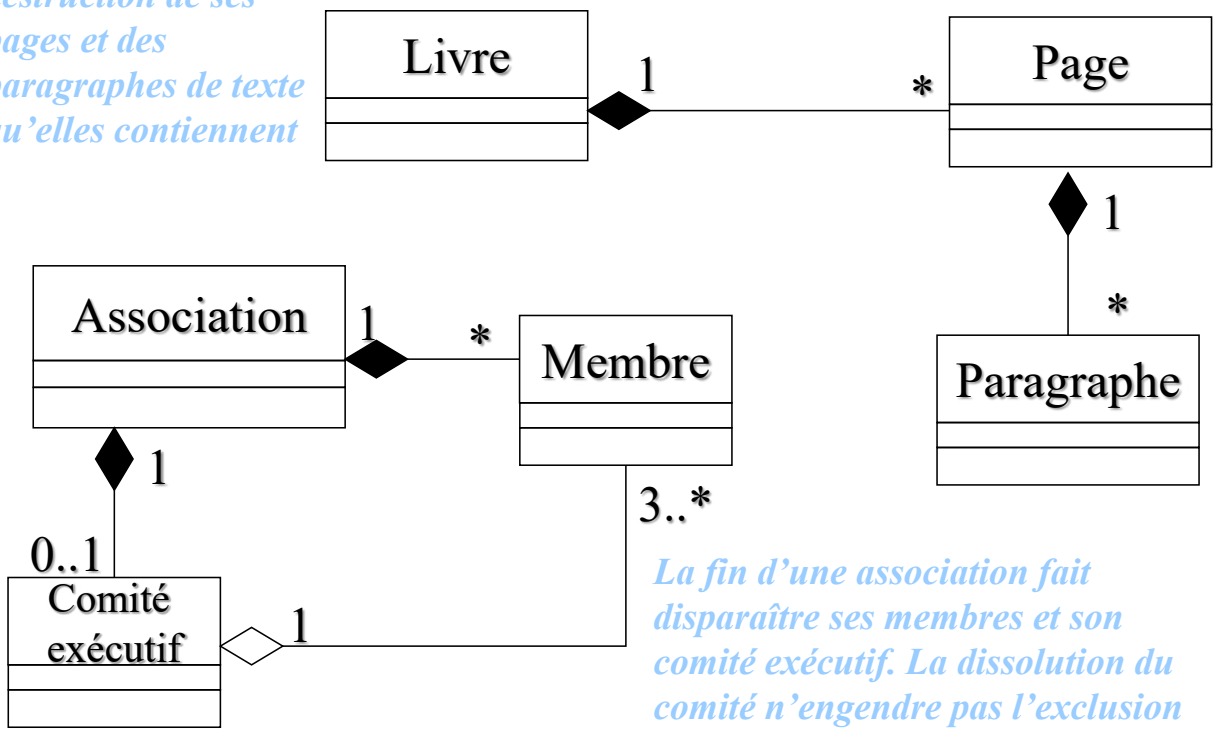
- Est une **agrégation forte** entre deux instances (un composite et un composant)
- Relation d'inclusion de type **« est le possesseur exclusif de »** ou **« est composé (physiquement) de »**
- Exprime une dépendance existentielle : si l'élément composite est détruit (ou copié), ses éléments composants le sont aussi. Les éléments composants peuvent cependant être détruits avant le composite.
- A un instant donné, un élément composant ne peut être lié qu'à un seul élément composite.



VII.5 Autres concepts du diagramme de classes UML - Association (4)

Exemples composition/agrégation

La destruction d'un livre entraîne la destruction de ses pages et des paragraphes de texte qu'elles contiennent

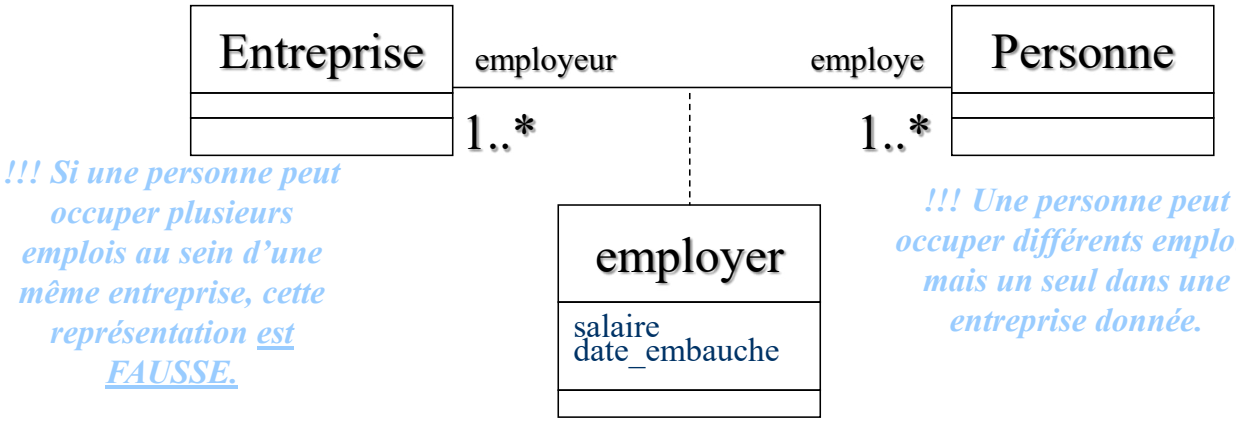


La fin d'une association fait disparaître ses membres et son comité exécutif. La dissolution du comité n'engendre pas l'exclusion de ses membres.

VII.6 Autres concepts du diagramme de classes UML - Association (5)

Association réifiée

- Chaque instance de la classe d'association est associée à un lien unique entre deux instances de classe. Si ce n'est pas le cas, il faut soit ajouter une autre dimension au niveau de l'association (exemple une dimension temporelle), soit **réifier** l'association.



!!! Si une personne peut occuper plusieurs emplois au sein d'une même entreprise, cette représentation est FAUSSE.

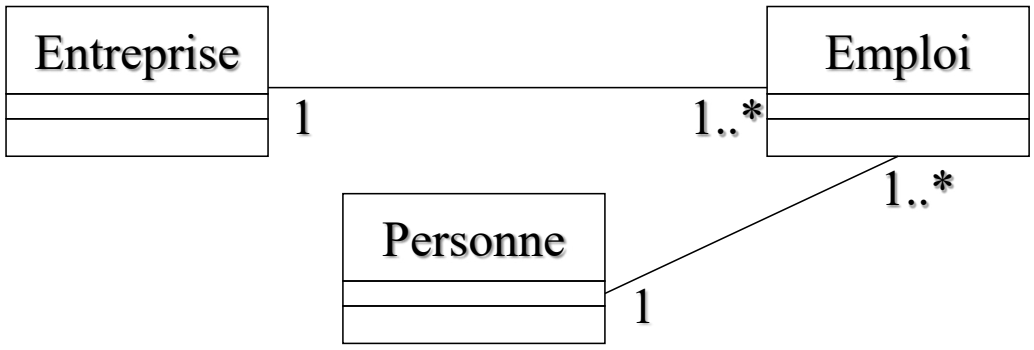
!!! Une personne peut occuper différents emplois mais un seul dans une entreprise donnée.

L'association Employer entre une entreprise et une personne possède comme propriétés le salaire et la date d'embauche.

VII.6 Autres concepts du diagramme de classes UML - Association (6)

Association réifiée

Réifier = transformer les concepts abstraits en réalités concrètes.



L'association entre Personne et Entreprise a été **réifiée** (introduction d'une classe intermédiaire) pour exprimer «concrètement» le fait qu'une personne peut occuper plusieurs emplois dans la même entreprise.